

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите
Руководитель ОП
_____ А.В. Щукин
« _____ » _____ 202_ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
РАБОТА БАКАЛАВРА
РАЗРАБОТКА ЛОКАЛЬНОГО ЛЕГКОВЕСНОГО ХРАНИЛИЩА НА БАЗЕ
LSM-ДЕРЕВЬЕВ ДЛЯ АВТОНОМНЫХ JAVA-ПРОЕКТОВ
по направлению подготовки 09.03.03 Прикладная информатика
Направленность (профиль) 09.03.03_03 Интеллектуальные инфокоммуникацион-
ные технологии

Выполнил
студент гр. 5130903/10302

Д.Н. Виниченко

Руководитель
Старший преподаватель ВШ ПИ

Е.Е. Андрианова

Консультант
по нормоконтролю

Е.Е. Андрианова

Санкт-Петербург
2025

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

Руководитель ОП

_____ А.В. Щукин

« _____ » _____ 202_ г.

ЗАДАНИЕ
на выполнение выпускной квалификационной работы

студенту Виниченко Даниилу Николаевичу, гр. 5130903/10302

1. Тема работы: «Разработка локального легковесного хранилища на базе LSM-деревьев для автономных Java-проектов».
2. Срок сдачи студентом законченной работы: 17.05.2025.
3. Исходные данные по работе: нет.
4. Содержание работы (перечень подлежащих разработке вопросов):
 - 4.1. Анализ предметной области. Обзор подходов организации данных в базах данных.
 - 4.2. Обзор и сравнение инструментальных средств или технологий имплементаций хранилищ на основе LSM-деревьев.
 - 4.3. Проектирование собственного решения, реализующего LSM-дерево для хранения данных.
 - 4.4. Практическая реализация решения.
 - 4.5. Тестирование разработанного приложения. Сравнение с похожими существующими решениями, реализующими LSM-деревья.
5. Перечень графического материала (с указанием обязательных чертежей):
 - 5.1. Диаграмма классов.
 - 5.2. Диаграмма компонентов.
6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии (при наличии):

- 6.1. Язык программирования Java 8.
- 6.2. IDE IntelliJ IDEA.
- 6.3. Среда тестирования JUnit Jupiter.
- 6.4. Язык Golang 1.24 и библиотека BadgerDB.
- 6.5. Язык C++ 17 и библиотека RocksDB.
- 6.6. Текстовый редактор VS Code.
- 7. Консультанты по работе:
 - 7.1. Старший преподаватель ВШ ПИ, Е.Е. Андрианова.
 - 7.2. Старший преподаватель ВШ ПИ, Е.Е. Андрианова (нормоконтроль).
- 8. Дата выдачи задания: 12.12.2024.

Руководитель ВКР _____ Е.Е. Андрианова

Консультант _____ Е.Е. Андрианова

Задание принял к исполнению 12.12.2024

Студент _____ Д.Н. Виниченко

РЕФЕРАТ

На 68 с., 12 рис., 17 табл., 1 прил.

Ключевые слова: NoSQL, Базы данных, LSM-деревья, Автономные встроенные хранилища, Apache Casandra, RocksDB, BadgerDB, Java, Автономные устройства, SSTable,

Тема выпускной квалификационной работы: «Разработка локального легковесного хранилища на базе LSM-деревьев для автономных Java-проектов».

В этой работе исследуются современные подходы к организации эффективных локальных NoSQL-хранилищ на основе LSM-деревьев, с фокусом на Apache Cassandra и RocksDB. Объектом исследования выступают алгоритмы MemTable→SSTable и методы компактации. Цель работы – провести обзор существующих архитектур, выявить их преимущества и ограничения, а затем создать Java-библиотеку без внешних зависимостей, оптимизированную для встраиваемых систем и IoT-устройств. Методология включает теоретический анализ LSM-деревьев, сравнительный обзор стратегий записи и компактации, проектирование и реализацию прототипа на Java 8 с CRC-контрольной суммой и Bloom-фильтрами, а также комплексное тестирование: юнит-тесты и нагрузочные стенды на Java, Go+BadgerDB и C++17+RocksDB, в том числе в Docker-контейнере с лимитами 0,5 CPU и 512 MB RAM. По результатам нагрузочных тестов разработка продемонстрировала среднюю задержку записи до 12 μ s и чтения до 4 μ s при footprint менее 20 MB, а все проверки CRC и консистентности прошли на 100%. Полученный прототип готов к практическому применению в автономных системах с ограниченными ресурсами, таких как IoT-датчики и встроенные контроллеры.

ABSTRACT

On 68 pages, 12 figures, 17 tables, 1 appendices

Keywords: NoSQL, Data bases, LSM-trees, Local integrated storages, Apache Casandra, RocksDB, Java, Autonomous devices, SSTable

The subject of the graduate qualification work is «Development of a local lightweight storage based on LSM-trees for autonomous Java projects».

This work investigates modern approaches to building efficient local NoSQL storages based on LSM-trees, focusing on Apache Cassandra and RocksDB. The

subject of research is the MemTable→SSTable algorithm and compaction methods. The objective is to review existing architectures, identify their strengths and limitations, and then implement a minimal Java 8 library without external dependencies optimized for embedded systems and IoT devices. The methodology includes theoretical analysis of LSM-trees, comparative study of write and compaction strategies, design and implementation of a Java prototype with CRC validation and Bloom filters, and comprehensive testing: unit tests and load benchmarks on Java, Go+BadgerDB, and C++17+RocksDB, including in a Docker container constrained to 0.5 CPU and 512 MB RAM. Load testing results show write latencies up to 12 μ s and read latencies up to 4 μ s with a memory footprint under 20 MB, and 100% success in CRC and consistency checks. The prototype is ready for practical use in resource-constrained autonomous systems such as IoT sensors and embedded controllers, with potential for further optimization via sparse indexing and GC tuning.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	8
1 Анализ предметной области.....	10
1.1 Обзор подходов организации данных в базах данных	10
1.1.1 SQL.....	10
1.1.2 NoSQL	11
1.1.3 LSM-деревья	13
1.2 Обзор и сравнение инструментальных средств или технологий	15
1.2.1 Основы LSM-деревьев	15
1.2.2 Современные имплементации LSM-деревьев	18
1.2.3 Write-Ahead Log (WAL) в контексте LSM-деревьев	19
1.3 Обзор существующих решений и аналогов.....	21
1.3.1 Apache Cassandra.....	21
1.3.2 RocksDB	24
1.4 Вывод по главе	34
2 Проектирование прототипа хранилища	35
2.1 Функциональные и нефункциональные требования	35
2.2 Высокоуровневая архитектура решения	36
2.3 Интерфейс программной библиотеки	38
2.4 Формат хранения SSTable.....	39
2.5 Отказ от WAL и обеспечение отказоустойчивости.....	40
2.6 Компактация	41
2.7 Bloom-Filter	44
2.8 Манифест и метаданные.....	45
2.9 Последовательность операций при put и при сбое.....	46
2.10 Вывод по главе	46
3 Практическая реализация решения	47
3.1 Технологический стек	47
3.2 Структура репозитория и схема пакетов	47
3.3 Описание классов	49
3.4 Особенности реализации ключевых компонентов	50
3.5 Поддержка расширяемости	52
3.6 Потенциал развития.....	52
3.7 Вывод по главе	53
4 Тестирование и апробация реализованного решения	54

4.1 Введение в автономные системы и ограниченные ресурсы	54
4.2 Среда тестирования и инструменты	55
4.2.1 Аппаратная платформа	55
4.2.2 Программная платформа	55
4.3 Unit-тестирование как проверка корректности и устойчивости	56
4.3.1 Тест-кейсы для операций put/get/delete	56
4.3.2 Тестирование CRC-проверки и механизма переименования	57
4.3.3 Тесты ManifestHandler	57
4.4 Нагрузочное тестирование: пропускная способность и потребление ресурсов	58
4.4.1 MixedTest (Java 8)	58
4.4.2 PutOnlyTest (Java 8)	58
4.4.3 Go + BadgerDB	59
4.4.4 C++ + RocksDB	60
4.4.5 Выводы по нагрузочному тестированию	60
4.5 Тестирование в ограниченной среде	61
4.5.1 Конфигурация тестового стенда	61
4.5.2 Анализ полученных результатов	62
4.5.3 Сравнение с "мощной" средой	63
4.5.4 Анализ адаптации JVM	64
4.6 Выводы по главе	65
ЗАКЛЮЧЕНИЕ	66
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	67
Приложение 1 Код приложения	69

ВВЕДЕНИЕ

В последние годы можно заметить рост объёма генерируемых и обрабатываемых данных в автономных и встраиваемых системах: от IoT-устройств до мобильных приложений и edge-агентов. Традиционные клиент–серверные СУБД не всегда способны удовлетворить требования к высокой скорости записи, малому потреблению ресурсов и автономности работы без постоянного соединения.

В данной работе исследуются принципы построения и практические реализации таких хранилищ на основе LSM-деревьев: от архитектуры MemTable → SSTable с контролем производимых операций записи до механизмов Bloom-фильтров, компактации и текстового манифеста. Анализируются существующие решения (RocksDB, Apache Cassandra) и описываются собственные наработки на Java и определяется круг задач, решаемых предлагаемым прототипом.

Актуальность исследования подкрепляется тем, что современные автономные системы, такие как носимые устройства, умные датчики, автономные агенты и мобильные приложения, требуют локальных хранилищ с высокой пропускной способностью записи и минимальными требованиями к ресурсам. LSM-деревья уже активно применяются в распределённых СУБД, те же Cassandra и RocksDB, однако их можно адаптировать и под более легковесные решения.

Предметом исследования являются архитектурные и алгоритмические подходы к реализации LSM-хранилищ в условиях ограниченных ресурсов: структура данных MemTable, SSTable файлы с CRC и индексами, алгоритмы tier-компактации, Bloom-фильтры и управление манифестом.

Цель работы - спроектировать и реализовать на Java лёгкое и высокопроизводительное встроенное хранилище на основе LSM-деревьев, способное эффективно работать в автономных системах с ограниченной памятью и I/O.

Теоретическая и методологическая база включает классические работы по LSM-деревьям (O’Neil et al., Acta Informatica, 1996), исследования Bloom-фильтров (Broder & Mitzenmacher, 2002), современные статьи по tier-компактации (Luo & Carey, VLDB Journal, 2020), а также документация к Java 8-ой версии. Методологию составляют анализ открытых решений, проектирование архитектуры и прототипирование в Java.

Информационная база. Основные источники информации:

– Официальная документация RocksDB, Apache Cassandra, BadgerDB и Java.

- Научные статьи по LSM-деревьям, Bloom-фильтрам и средствам компактации.
- Обзоры и презентации, подготовленные в рамках работы.

Степень научной разработанности проблемы следующая - LSM-деревья и связанные техники активно изучаются в контексте крупных СУБД, однако вопросы их упрощённой адаптации, отказоустойчивости без WAL и динамической компактации в условиях ограниченных ресурсов остаются мало исследованными.

Научная новизна основана на следующих свойствах реализуемого в ходе проекта. В работе предлагается:

- Отказ от WAL в пользу атомарного сброса MemTable → SSTable.temp → rename с CRC;
- Интегрированная size-tiered компактация и Bloom-фильтры для ускорения запросов;
- Текстовый манифест с контролем целостности и автоматическим восстановлением;
- Чистая Java-реализация без сторонних зависимостей.

Практическая значимость - разработанный прототип может быть включён в проекты автономных устройств и edge-агентов, где критичны высокая скорость записи и экономия ресурсов. Полученные результаты станут основой для дальнейшей оптимизации и промышленного применения лёгких NoSQL-хранилищ.

Теоретическая значимость работы отражается, прежде всего, в:

- исследовании и обосновании гипотезы о том, что отказ от классического WAL в пользу атомарных операций записи (snapshot → .temp → rename) при сохранении контроля целостности через CRC обеспечивает равнозначную отказоустойчивость при существенно сниженных I/O-затратах;
- развитии методик проектирования и оценки производительности лёгких встраиваемых хранилищ на основе LSM-деревьев, включая формализацию подходов к компактации и интеграции механизмов поиска;
- формализации требований к встроенным системам хранения данных с ограниченными ресурсами, что в будущем позволит систематизировать и унифицировать архитектурные решения для IoT- и edge-платформ.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор подходов организации данных в базах данных

История развития баз данных демонстрирует широкий спектр подходов к организации и хранению данных. Помимо доминирующей в современной индустрии реляционной модели баз данных (РМБД), ранее существовали альтернативные подходы, такие как сетевые и иерархические модели. Однако эти модели не смогли выдержать испытание временем, уступив место реляционной модели, которая на момент своего появления обозначила новый этап в развитии технологий управления данными. Реляционная модель, благодаря своей универсальности, гибкости и теоретической строгости, вытеснила своих предшественников, став основой для подавляющего большинства современных систем управления базами данных.

Предполагалось, что реляционная модель баз данных рано или поздно столкнётся с конкурирующим подходом, способным занять её место. В конце XX - начале XXI века на эту роль прочили модели NoSQL (Not Only SQL). Однако полного вытеснения реляционных баз данных не произошло. Тем не менее, баланс на рынке систем управления данными значительно изменился. NoSQL базы продемонстрировали свою эффективность в решении определённых задач, что привело к появлению гибридных систем хранения данных, сочетающих возможности реляционных и нереляционных подходов¹.

1.1.1 SQL

SQL (Structured Query Language)² - язык для работы с данными в реляционных СУБД. Основан на реляционной модели, обеспечивающей связь таблиц через первичные и внешние ключи, индексы.

Архитектура:

- **Физическая:** Определяет, как данные хранятся на файловой системе (страницы, файлы, журналы).

¹Клеппман М. *Высоконагруженные приложения. Программирование, масштабирование, поддержка*. – Москва: ДМК Пресс, 2022. – 576 с.

²Sharma V., Dave M. *SQL and NoSQL databases* // International Journal of Advanced Research in Computer Science and Software Engineering. 2012. Т. 2, № 8. URL: https://www.researchgate.net/profile/Meenu-Dave/publication/303856633_SQL_and_NoSQL_Databases/links/5758557f08ae9a9c954a7573/SQL-and-NoSQL-Databases.pdf

- **Логическая:** Включает таблицы, представления, процедуры, функции и триггеры.

Свойства ACID:

- **Атомарность:** Все операции транзакции выполняются или ни одна.
- **Согласованность:** Данные сохраняют корректное состояние.
- **Изоляция:** Транзакции выполняются независимо.
- **Надежность:** Изменения сохраняются даже при сбоях.

SQL в Big Data: SQL предоставляет мощные аналитические инструменты, но требует четкой схемы данных, что ограничивает его применение для обработки непредсказуемых и неструктурированных данных.

1.1.2 NoSQL

NoSQL - это система управления данными, ориентированная на горизонтальное масштабирование и гибкость. Включает базы данных типов: документные, графовые, key-value, гибридные и другие.

Модели данных в NoSQL:

- **Key-value:** Данные хранятся в парах ключ-значение. Примеры: Redis, RocksDB, BadgerDB.
- **Документные:** Данные хранятся в виде документов (JSON). Примеры: MongoDB, RavenDB.
- **Графовые:** Описывают связи между данными. Примеры: Neo4J, HypergraphDB.

Принципы NoSQL:

- **ACID-free:** NoSQL базы данных отказываются от концепции ACID, поскольку они могут привести к снижению производительности, особенно при обеспечении изоляции и атомарности..
- **BASE:** Доступность, мягкое состояние, финальная согласованность.

CAP-теорема:

- **Согласованность:** Данные на всех машинах одинаковы.
- **Доступность:** Данные всегда доступны для пользователей.
- **Устойчивость к разделению:** База данных продолжает работать при сбоях в сети.

Сравнение SQL и NoSQL

NoSQL пока находится на стадии становления и имеет ряд недостатков:

- Системы менее функциональны и стабильны, чем SQL.
- Ограниченная экспертиза в области NoSQL.
- BASE не обеспечивает такой же надежности, как ACID, что важно для критически важных данных (например, банковских операций).

На таблице 1.1 приведено сравнение SQL и NoSQL подходов.

Таблица 1.1

Сравнение SQL и NoSQL^{1,2}

Критерий	SQL	NoSQL
Структура данных	Чёткая, заранее определённая схема	Гибкая структура, возможна без схемы
Масштабирование	Вертикальное (масштабирование узла)	Горизонтальное (добавление узлов)
Подходящие задачи	Транзакционные системы (например, банковские)	Большие объёмы данных, Big Data, распределённые системы
Модель согласованности	ACID (жёсткая согласованность)	BASE (гибкая, в eventual consistency)
Типичные области применения	ERP-системы, CRM, бухгалтерия	IoT, аналитика, хранение JSON, логов и др.

BASE

BASE - это концепция NoSQL баз данных, гарантирующая их надежность даже при утрате строгой согласованности. На таблице 1.2 приведено сравнение принципов BASE с принципами ACID, которые соблюдаются в SQL базах данных.

BASE расшифровывается как:

- **Basically Available:** Система обеспечивает доступность данных в любой момент.

¹Sethi S., Panda S. *SQL or NoSQL—Practical Aspect and Rational behind Choosing Data Stores* // Journal of Computer and Communications. 2024. Т. 12, № 8. С. 155–174. URL: <https://www.scirp.org/journal/paperinformation?paperid=135134>

²Binani S., Gutti A., Upadhyay S. *SQL vs. NoSQL vs. NewSQL — a comparative study* // Database. 2016. Т. 6, № 1. С. 1–4. URL: <https://www.caeaccess.org/archives/volume6/number1/binani-2016-cae-652418.pdf>

- **Soft state:** Состояние системы может изменяться даже без новых вводимых данных.
- **Eventually Consistent** Данные становятся согласованными со временем, так как система обновляет их в нужных местах.

Таблица 1.2

Сравнение подходов ACID и BASE

Критерий	ACID	BASE
Расшифровка аббревиатуры	Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventually consistent
Основной приоритет	Согласованность и надёжность данных	Доступность и устойчивость к сетевым сбоям
Тип согласованности	Жёсткая, строгая согласованность	Гибкая, eventual consistency
Подход к отказоустойчивости	Пессимистичный (избегать конфликтов)	Оптимистичный (разрешать конфликты постфактум)
Сложность реализации	Требует сложной транзакционной поддержки	Простая архитектура, высокая производительность
Типичные сферы применения	Банковские системы, финансы, критичные данные	Веб-приложения, кэш-системы, NoSQL-базы

NoSQL лучше подходит для Big Data благодаря своей масштабируемости и гибкости, но для критически важных данных, например в банковской сфере, SQL более предпочтителен.

1.1.3 LSM-деревья

LSM-деревья (Log-Structured Merge-Trees)³ - это структура данных, оптимизированная для высокоскоростного записи и работы с большими объемами данных. Они особенно полезны в системах хранения, таких как RocksDB, благодаря их способности минимизировать количество операций записи на диск. На табли-

³O'Neil P. et al. The log-structured merge-tree (LSM-tree) // Acta Informatica. – 1996. – Т. 33. – С. 351–385. URL: <https://www.inf.ufpr.br/eduardo/ensino/ci763/papers/lsmtree.pdf>

цах 1.3, 1.4 и 1.5 приведён процесс поэтапной перезаписи данных с MemTable в SSTable.

Таблица 1.3

А. Добавление данных в память (MemTable)

Key	Value	Status
K1	V1	new
K2	V2	new
K3	V3	new

Когда MemTable заполняется, она преобразуется в SSTable (см. таблицу 1.4):

Таблица 1.4

В. Флашинг на диск (создание SSTable)

Key	Value	Source
K1	V1	flushed from MemTable
K2	V2	flushed from MemTable
K3	V3	flushed from MemTable

После добавления новых данных и обновлений выполняется слияние (см. таблицу 1.5):

Таблица 1.5

С. Слияние SSTables

Key	Value	Status
K1	V4	updated
K2	V2	no changes
K3	V3	no changes
K4	V5	new key

LSM-деревья поддерживают высокую скорость записи за счёт буферизации данных в памяти и постепенного упорядочивания на диске.

1.2 Обзор и сравнение инструментальных средств или технологий

1.2.1 Основы LSM-деревьев

LSM-дерево⁴, или Log-Structured Merge-tree, представляет собой структуру данных, разработанную для повышения производительности записи. В отличие от традиционных структур, таких как B-дерево, LSM-дерево использует подход «обновления вне места». Это означает, что новые данные записываются в новые области памяти, вместо того чтобы перезаписывать существующие записи. Такой подход снижает нагрузку на ввод-вывод за счёт использования последовательных операций записи, что делает LSM-деревья особенно полезными для систем, где преобладают частые записи.

Разница между подходами «на месте» и «вне места» значительно влияет на поведение системы. Например, B-деревья модифицируют существующие записи при обновлении, что обеспечивает высокую скорость чтения, так как всегда доступна самая последняя версия данных. Однако этот метод сопровождается высокими затратами на запись из-за случайных операций ввода-вывода. Кроме того, частые изменения могут фрагментировать страницы индекса, что снижает эффективность использования пространства.

LSM-деревья, напротив, добавляют новые данные в конец лога, сохраняя старые записи нетронутыми, что позволяет минимизировать затраты на запись и упростить процессы восстановления, так как предыдущие данные всегда остаются доступными. Однако обратной стороной является снижение производительности чтения: поскольку данные одного ключа могут быть разбросаны по разным компонентам, приходится выполнять дополнительные операции для их объединения (полное сравнение приведено в таблице 1.6).

Сравнение обновления «на месте» и «вне места»

Обновление «на месте» (например, B-деревья):

Операция: Перезаписывает старые записи для внесения изменений (например, обновление $(k1, v1) \rightarrow (k1, v4)$).

Преимущества:

⁴O'Neil P. et al. The log-structured merge-tree (LSM-tree) // Acta Informatica. – 1996. – Т. 33. – С. 351–385.
URL: <https://www.inf.ufpr.br/eduardo/ensino/ci763/papers/lsmtree.pdf>

- Оптимизация **чтения**: хранится только самая последняя версия каждой записи.

Недостатки:

- Случайные операции ввода-вывода при обновлениях снижают **производительность записи**.
- Обновления и удаления фрагментируют страницы индекса, снижая **эффективность использования пространства**.

Обновления «вне места» (например, LSM-деревья)

Операция: Добавляет новые обновления в последовательный лог (например, запись $(k1, v4)$ в новое место).

Преимущества:

- Быстрые записи благодаря **последовательному вводу-выводу**.
- Упрощение восстановления данных за счёт сохранения старых данных.

Недостатки:

- Медленное чтение, так как данные могут находиться в нескольких местах.
- Требуются фоновые процессы для реорганизации данных и повышения эффективности.

Исторически подход «обновлений вне места» появился ещё в 1970-х годах с введением дифференциальных файлов. Эта идея была развита в 1980-х в лог-структурированных базах данных, таких как Postgres, где данные записывались в последовательный лог, а фоновые процессы занимались сбором устаревших записей.

Сравнение структур данных B-tree и LSM-tree

В 1996 году была предложена концепция LSM-деревьев, которая решала эти проблемы. Вместо простого добавления записей в лог использовался механизм слияния, встроенный в структуру дерева. LSM-дерево состоит из последовательности компонентов, где начальный компонент C_0 хранится в оперативной памяти и обрабатывает новые записи, а остальные $C_1, C_2, \dots$ - на диске. Когда C_0 заполняется, его данные сливаются с C_1 , а затем с последующими уровнями. Этот процесс позволяет достичь стабильного соотношения между производительностью записи, чтения и эффективностью использования пространства. Процесс слияния данных из памяти в постоянное хранилище изображен на рисунке 1.1.

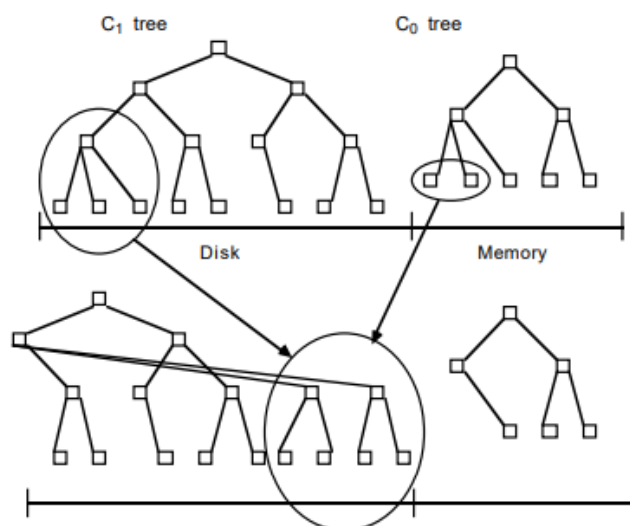
В таблице 1.6 приведено наглядное сравнение B-деревьев и LSM-деревьев на основе определенных критериев.

Таблица 1.6

Сравнение структур данных B-tree и LSM-tree

Критерий	B-tree	LSM-tree
Производительность операций чтения	Высокая, особенно для точечного поиска	Средняя, так как данные могут быть распределены по уровням
Производительность операций записи	Средняя: операции записи сразу отражаются на дереве, что требует постоянного обновления структуры	Высокая: записи буферизуются в памяти и периодически сливаются в хранилище
Эффективность использования дискового пространства	Средняя: требует больше дискового пространства из-за необходимости хранения индексов	Высокая: слияние данных снижает фрагментацию и улучшает использование пространства
Масштабируемость	Ограниченная: с ростом данных производительность снижается	Высокая: эффективно обрабатывает большие объёмы данных благодаря слиянию уровней
Подходящие сценарии использования	Подходит для систем, где преобладают операции чтения	Подходит для систем с интенсивной записью и требованием высокой пропускной способности
Затраты на поддержание структуры	Низкие: структура остаётся относительно стабильной	Высокие: требует ресурсов для периодического слияния уровней

Интересно, что параллельно с LSM-деревьями были предложены альтернативные подходы, такие как ступенчатое слияние. Этот метод также был направлен на оптимизацию записи, но с использованием иной стратегии объединения данных. Многие идеи этих ранних разработок нашли отражение в современных реализациях LSM-деревьев.

Рис.1.1. Визуализация слияния⁵

1.2.2 Современные имплементации LSM-деревьев

Современные LSM-деревья⁶ продолжают использовать подход «обновлений вне места», чтобы минимизировать затраты на случайные операции ввода-вывода. Новые записи сначала помещаются в память, где обрабатываются с использованием высокопроизводительных структур, таких как skip-list карты. В то же время, удаление данных добавляет так называемые «антиматерии» - специальные записи, которые указывают на удаление ключа, но сохраняются в системе до завершения всех процессов слияния.

Одним из важных нововведений стало использование неизменяемых компонентов на диске. Это позволяет значительно упростить управление конкурентным доступом и процессы восстановления. Вместо того чтобы изменять существующие компоненты, данные объединяются в новые структуры, что избавляет от необходимости блокировать или изменять старые файлы.

Диск, как правило, организован с использованием B-деревьев или SSTable (Sorted-String Table). SSTable состоит из упорядоченных по ключам блоков данных и индексов, которые содержат диапазоны ключей для быстрого поиска. Такая структура позволяет эффективно обрабатывать запросы, несмотря на многослойность LSM-дерева.

⁵O'Neil P. et al. The log-structured merge-tree (LSM-tree) // Acta Informatica. – 1996. – Т. 33. – С. 351–385. URL: <https://www.inf.ufpr.br/eduardo/ensino/ci763/papers/lsmtree.pdf>

⁶Luo C., Carey M. J. *LSM-based storage techniques: a survey* // The VLDB Journal. 2020. Т. 29, № 1. С. 393–418. URL: <https://link.springer.com/article/10.1007/s00778-019-00555-y>

Запросы в LSM-деревьях обрабатываются через последовательный поиск данных в нескольких компонентах, начиная с самых новых. Точечный поиск, например, завершится, как только будет найдена первая подходящая запись. Для диапазонных запросов результаты объединяются с использованием очереди с приоритетом, чтобы гарантировать доступ к последним версиям данных.

Однако накопление дисковых компонентов со временем может ухудшить производительность чтения. Чтобы этого избежать, данные периодически сливаются, сокращая общее количество компонентов. Для этого используются две основные стратегии: выравнивающее и ступенчатое слияние.

Выравнивающее слияние оптимизирует чтение, так как на каждом уровне хранится только один компонент. Данные последовательно объединяются, чтобы на следующем уровне оказывались более крупные компоненты. В отличие от этого, **ступенчатое слияние** допускает наличие нескольких компонентов на одном уровне. Объединение происходит только тогда, когда уровень заполняется, что снижает нагрузку на запись. Выбор подходящей стратегии зависит от потребностей системы: выравнивание подходит для приложений с частыми чтениями, а ступенчатое слияние - для сценариев с преобладанием записей.

1.2.3 Write-Ahead Log (WAL) в контексте LSM-деревьев

Write-Ahead Log (WAL) - это механизм логирования операций, используемый в разного рода системах, в том числе и на основе LSM-деревьев, для обеспечения надежности данных. Перед записью данных в память (MemTable) или на диск (SSTable) операции фиксируются в WAL, что позволяет восстановить данные после сбоя.

WAL представляет собой последовательный лог-файл на диске, куда записываются все изменения данных. Запись в WAL выполняется до обновления MemTable, что гарантирует сохранение данных даже при неожиданном сбое системы. Записи в WAL выполняются последовательно, минимизируя накладные расходы на операцию ввода-вывода.

Как работает WAL

1. **Запись данных:** Когда приходит новый запрос (например, `put(key, value)`):
 - Операция сначала записывается в WAL.

- Затем данные помещаются в MemTable - структуру в оперативной памяти. На таблице 1.7 изображен примерный процесс записи в MemTable с предварительной записью данных в WAL.

Таблица 1.7

Пример записи данных в WAL и MemTable

Операция	WAL	MemTable
put(K1, V1)	K1 → V1 записано	K1 → V1 добавлено
put(K2, V2)	K2 → V2 записано	K2 → V2 добавлено

2. Флашинг на диск: Когда MemTable достигает предельного размера:

- Она преобразуется в SSTable и сохраняется на диск.
- WAL может быть очищен, так как данные уже безопасно записаны в долговременное хранилище. На таблице 1.8 изображен процесс перезаписи данных из MemTable в SSTable с очисткой WAL от логов записи данных в память.

Таблица 1.8

Пример флашинга данных из MemTable в SSTable

SSTable Level 0	WAL	MemTable
K1 → V1, K2 → V2	Очищен	Пустой

3. Восстановление после сбоя: Если система неожиданно завершила работу, при перезапуске:

- WAL считывается для восстановления недописанных данных в MemTable.
- После восстановления работа продолжается, и данные сохраняются в SSTable.

Преимущества WAL

Надежность: Обеспечивает защиту от потери данных при сбоях.

Последовательность: Упрощает процесс восстановления, так как все операции фиксируются в строгом порядке.

Ограничения WAL

Накладные расходы. Несмотря на эффективность последовательной записи, наличие WAL увеличивает общее количество операций записи.

Удаление старых данных. Для предотвращения роста размера WAL, данные регулярно очищаются, когда они становятся ненужными.

1.3 Обзор существующих решений и аналогов

1.3.1 *Apache Cassandra*

Apache Cassandra^{7,8} это нереляционная легковесная NoSQL база данных с открытым исходным кодом. Одним из самых важных свойств Cassandra является распределенность. Эта база данных легко масштабируется, что позволяет разрабатывать высоконагруженные приложения, а также предотвращать потерю данных при системных сбоях, например, выход из строя серверного оборудования.

Вариант использования Cassandra, можно сказать, идентичен классическим базам данных в крупных производствах, по типу PostgreSQL или Oracle - "большой" сервер базы данных, возможно распределенный (в случае с Cassandra - точно распределенный), к которому по сети приходят запросы на обработку или извлечение информации. Поэтому в контексте встроенных хранилищ рассматривать эту базу данных бессмысленно, не тот случай. Однако можно рассмотреть её механизм хранения.

Поскольку это распределенная база данных, Cassandra может иметь (и обычно имеет) несколько узлов. Узел представляет собой одну реплику Cassandra. Эти узлы взаимодействуют друг с другом через протокол, называемый **gossip**, который представляет собой процесс компьютерной одноранговой связи. Cassandra также имеет архитектуру без единого центра принятия решений, где любой узел в базе данных может предоставлять точно такую же функциональность, как и любой другой узел. Несколько узлов могут быть логически организованы в кластеры или "кольцо".

⁷Apache Cassandra Documentation. URL: <https://cassandra.apache.org/doc/latest/>

⁸Luo C., Carey M. J. *LSM-based storage techniques: a survey* // The VLDB Journal. 2020. Т. 29, № 1. С. 393–418. URL: <https://link.springer.com/article/10.1007/s00778-019-00555-y>

Одна из причин популярности Cassandra заключается в том, что она позволяет разработчикам динамически масштабировать свои базы данных, используя готовое оборудование, без простоев.

1.3.1.1 Механизм Хранения

Механизм хранения Cassandra оптимизирован для высоконагруженных рабочих процессов, ориентированных на запись. Архитектура основана на ранее описанных **LSM-деревьях**. Хотя путь записи высоко оптимизирован, он идет на компромиссы с точки зрения производительности чтения и усиления записи. Для улучшения операций чтения Cassandra использует фильтры Блума при доступе к данным из SSTable.

Фильтр Блума - это вероятностная структура данных, используемая для проверки принадлежности элемента множеству. Он быстро отвечает на вопрос: "Может ли элемент быть в множестве?"

Фильтр состоит из битового массива и набора хэш-функций. При добавлении элемента хэш-функции вычисляют несколько индексов в массиве, которые устанавливаются в '1', что изображено на рисунке 1.2. При проверке наличия элемента хэшируются те же индексы: если хотя бы один бит равен '0', элемент точно отсутствует, это видно на рисунке 1.3, где в массиве находится лишь один хэш из трёх; если все равны '1', элемент может быть в множестве, но возможны ложные срабатывания, на рисунке 1.4 хэши "несуществующего" объекта всё равно находятся в массиве хэшей, что провоцирует ложно положительное срабатывание.

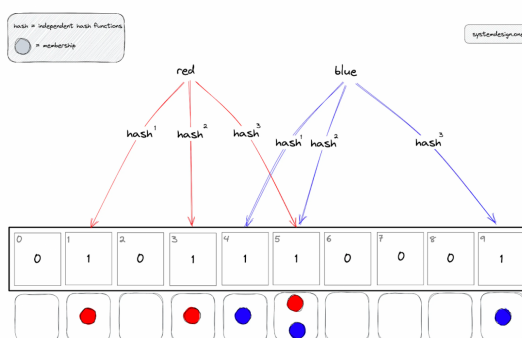


Рис.1.2. Добавление хэша в массив⁹

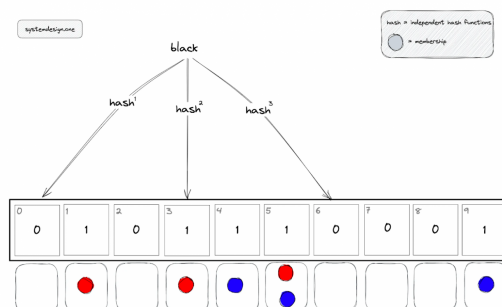


Рис.1.3. Истинно отрицательное срабатывание

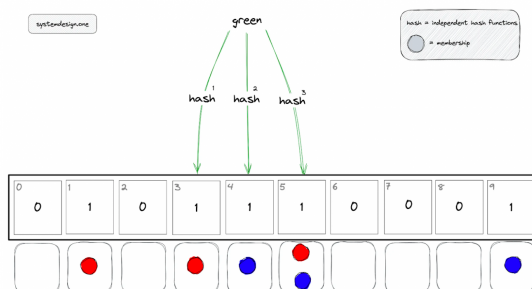
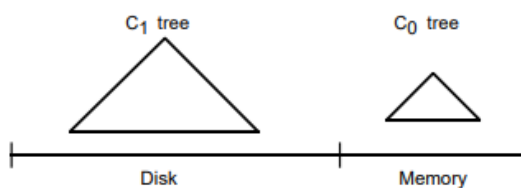


Рис.1.4. Ложно положительное срабатывание

Вероятность ложного срабатывания можно уменьшить, если предоставить фильтру больше рабочей памяти.

В Cassandra фильтр Блума применяется для минимизации затрат на чтение: он проверяет, хранит ли сегмент SSTable нужные данные, что позволяет избежать обращения к диску для заведомо пустых запросов.

Ядро хранилища состоит из memtables для данных в памяти и неизменяемых SSTables на диске, схема этих компонентов изображена на рисунке 1.5. Данные в SSTable хранятся уже отсортированными для обеспечения эффективной сортировки слиянием во время переноса записей из ОЗУ на ПЗУ. Кроме того, журнал предварительной записи (WAL), называемый журналом фиксации, обеспечивает устойчивость к сбоям и восстановлению транзакций.

Рис.1.5. Схема компонент¹⁰

⁹URL: <https://habr.com/ru/articles/788772/>

Последовательность шагов в пути записи:

- Регистрация данных в журнале логов
- Запись данных в memtable
- Очистка данных из memtable
- Хранение данных на диске в SSTables

1.3.2 RocksDB

RocksDB^{11,12} – это высокопроизводительный, устойчивый движок хранения данных в формате ключ-значение, созданный в 2012 году на основе кода LevelDB от Google. Он оптимизирован для специфических характеристик твердотельных накопителей (SSD), ориентирован на крупномасштабные (распределённые) приложения и разработан как библиотечный компонент, встраиваемый в приложения более высокого уровня. Каждый экземпляр RocksDB управляет данными только на устройствах хранения одного серверного узла; он не обрабатывает межузловые операции, такие как репликация и балансировка нагрузки, а также не выполняет операции высокого уровня, например контрольные точки (checkpoints). Реализация таких операций возлагается на приложение, но RocksDB предоставляет соответствующую поддержку для их эффективного выполнения.

RocksDB и его компоненты высоко настраиваемы, что позволяет адаптировать движок хранения к широкому спектру требований и нагрузок. Настройки могут включать обработку журнала предзаписи (WAL), стратегию сжатия данных и стратегию компактизации (процесс, который удаляет "мёртвые" данные и оптимизирует LSM-деревья, как описано). RocksDB можно настроить для обеспечения высокой пропускной способности записи, высокой скорости чтения, эффективности использования пространства или чего-то промежуточного.

Помимо использования в качестве движка хранения для баз данных, RocksDB применяется в следующих типах сервисов с весьма различающимися характеристиками:

- Обработка потоков данных (Stream processing)
- Сервисы логирования/очередей

¹⁰O'Neil P. et al. The log-structured merge-tree (LSM-tree) // Acta Informatica. – 1996. – Т. 33. – С. 351–385. URL: <https://www.inf.ufpr.br/eduardo/ensino/ci763/papers/lsmtree.pdf>

¹¹Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications // ACM Transactions on Storage (TOS). 2021. Т. 17, № 4. С. 1–32. URL: <https://dl.acm.org/doi/full/10.1145/3483840>

¹²Luo C., Carey M. J. *LSM-based storage techniques: a survey* // The VLDB Journal. 2020. Т. 29, № 1. С. 393–418. URL: <https://link.springer.com/article/10.1007/s00778-019-00555-y>

- Сервисы индексации
- Кэширование на SSD

За последнее десятилетие SSD-накопители получили широкое распространение для обслуживания данных веб-сервисов. Низкая задержка и высокая пропускная способность этих устройств не только ставят перед программным обеспечением задачу полного использования их возможностей, но и изменили подход к реализации многих сервисов, работающих с состоянием.

Высокая производительность SSD в ряде случаев сместила узкое место с ввода-вывода устройства на сеть как с точки зрения задержки, так и пропускной способности. Это сделало более привлекательным для приложений хранить данные на локальных SSD, а не использовать удаленные хранилища данных. Это увеличило спрос на движки хранилищ в формате "ключ-значение" которые можно встроить в приложения. RocksDB хорошо оптимизирован для удовлетворения этих требований.

RocksDB использует **LSM-деревья** в качестве основной структуры данных для хранения с поддержкой следующих ключевых операций:

Запись

При записи данных в RocksDB они добавляются в:

1. **Write Buffer (MemTable)** - буфер записи в оперативной памяти.
2. **Write Ahead Log (WAL)** - журнал предзаписи на диске, используемый для восстановления данных в случае сбоя.

MemTable реализуется в виде **skiplist**, чтобы поддерживать упорядоченность данных с сложностью операций вставки и поиска $O(\log n)$. WAL не является обязательным, но обеспечивает надежность данных.

Каждая SSTable хранит данные в отсортированном порядке, разделенные на блоки одинакового размера. SSTable является неизменяемой после записи. Для бинарного поиска каждая SSTable содержит индексный блок с одной записью индекса на каждый блок SSTable.

Skiplist

Skiplist (список с пропусками) - это структура данных, представляющая собой упорядоченный связный список с несколькими уровнями, которые позволяют ускорить поиск.

Основные свойства:

- **Быстрый поиск:** благодаря уровням можно "перепрыгивать" через множество элементов, сокращая сложность поиска с $O(n)$ (как в обычном списке) до $O(\log n)$.
- **Обновление:** вставка и удаление элементов выполняются локально, что упрощает работу по сравнению с деревьями.
- **Простота реализации:** skiplist легче реализовать, чем сбалансированные деревья (например, AVL или красно-чёрные деревья).

Skiplist часто используется в базах данных и кэшах, где важны скорость работы и простота реализации.

Когда размер MemTable достигает заданного предела:

- MemTable и WAL становятся неизменяемыми.
- Создаются новые MemTable и WAL для дальнейших записей.
- Содержимое MemTable сбрасывается в файл данных SSTable (Sorted String Table) на диск.
- Старая MemTable и связанный WAL удаляются.

Чтение

При чтении ключ сначала ищется в:

1. MemTable.
2. SSTable уровня 0.
3. SSTable последовательно более старых уровней, которые покрывают диапазон искомого ключа.

Для каждого случая используется бинарный поиск. Поиск продолжается до тех пор, пока ключ не будет найден или не станет ясно, что его нет на самом старом уровне.

Чтобы минимизировать операции ввода-вывода и накладные расходы на декомпрессию:

- **Горячие блоки SSTable** кэшируются в памяти.
- **Фильтры Блума** используются для исключения ненужных поисков в SSTable.

1.3.2.1 Компактация (Compaction)

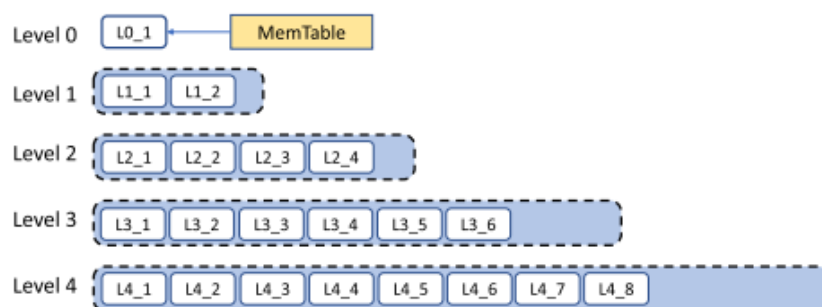


Рис.1.6. Уровни расположения SSTable¹³

LSM-дерево имеет несколько уровней, как это показано на рисунке 1.6. Новые SSTables создаются при сбросе MemTable и размещаются на уровне 0 (Level-0). Остальные уровни создаются процессом компактации.

Максимальный размер каждого уровня ограничивается параметрами конфигурации. Когда объём данных на уровне L превышает установленный предел:

1. Некоторые SSTables с уровня L выбираются.
2. Они объединяются с перекрывающимися SSTables на уровне $L + 1$.
3. Создается новая SSTable для уровня $L + 1$.

Во время компактации удаляются данные, которые были удалены или перезаписаны, а также оптимизируется производительность чтения и использование пространства. Этот процесс постепенно переносит данные из уровня 0 на последний уровень.

Компактация ввода-вывода эффективна, так как может выполняться параллельно и включает только массовое чтение и запись целых файлов.

Замечание: Уровни с большими номерами (например, уровень 2) называются более старыми уровнями, поскольку данные перемещаются вниз по дереву. SSTables на более старых уровнях могут покрывать неперекрывающиеся диапазоны ключей, а их блоки могут сжиматься для экономии дискового пространства.

¹³Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications // ACM Transactions on Storage (TOS). 2021. Т. 17, № 4. С. 1–32. URL: <https://dl.acm.org/doi/full/10.1145/3483840>

RocksDB поддерживает несколько типов компактации, что позволяет оптимизировать работу для разных сценариев использования:

Leveled Compaction

Этот метод компактации был адаптирован из LevelDB и улучшен. Каждому уровню задаются целевые размеры, увеличивающиеся экспоненциально (показаны пунктирными прямоугольниками на рисунке 1). Компакции запускаются проактивно, чтобы предотвратить превышение целевых размеров.

Tiered Compaction (Universal Compaction)

Этот стиль, похожий на подходы Apache Cassandra и HBase, использует ленивую компакцию. Несколько SSTables объединяются, если сумма файлов уровня 0 и файлов других уровней превышает заданный порог, или если отношение общего размера базы данных к размеру самого большого уровня превышает порог. Компакция откладывается до ухудшения производительности чтения или эффективности использования пространства, чтобы одновременно обработать больше данных.

FIFO Compaction

Устаревшие SSTables удаляются, как только размер базы данных достигает установленного лимита. Выполняются только лёгкие компакции. Этот стиль ориентирован на приложения, использующие кэширование в памяти.

Возможность выбора стиля компактации позволяет RocksDB поддерживать широкий спектр сценариев:

- **Leveled Compaction** обеспечивает баланс между производительностью записи и чтения.
- **Tiered Compaction** улучшает производительность записи и уменьшает усиление записи, но вредит производительности чтения.
- **FIFO Compaction** подходит для сценариев с преимущественной нагрузкой на запись (например, логирование или потоковая обработка).

Пример: Сервисы логирования или потоковой обработки могут использовать конфигурации, оптимизированные для записи, в то время как базы данных требуют сбалансированного подхода.

1.3.2.2 Работа с WAL (Write Ahead Log)

Для обеспечения долговечности данных традиционные базы данных фиксируют каждую запись в **WAL**. Однако для распределённых систем это менее критично. Репликация в распределённых системах обеспечивает восстановление данных из других реплик. Такие системы часто имеют собственные журналы репликации (например, **Paxos logs**), из-за чего WAL RocksDB может не понадобиться.

RocksDB предоставляет несколько режимов работы с WAL:

1. **Синхронная запись в WAL** (для максимальной гарантии долговечности).
2. **Буферизованная запись в WAL** (периодическая запись в фоновом режиме с минимальным влиянием на задержки запросов).
3. **Отключение WAL** (для сценариев с репликацией).

1.3.2.3 Автономность RocksDB

RocksDB часто используется в автономных приложениях, то есть в таких приложениях, которые не обладают какой-то инфраструктурой вокруг себя, будь то сервера баз данных или брокеры сообщений. Другими словами, это такие приложения, которые работают на одном узле, не зависят от распределённых систем и полностью управляют своим локальным хранилищем данных.

Как RocksDB работает в автономных приложениях

1. **Интеграция с приложением:** RocksDB предоставляется в виде библиотеки, которая включается напрямую в приложение. Вся логика взаимодействия с данными - управление вставками, удалениями, сканированием и оптимизация запросов - обрабатывается внутри самого приложения, используя RocksDB API.
2. **Хранение данных:** Данные сохраняются в локальной файловой системе.
3. **Обеспечение отказоустойчивости:** RocksDB гарантирует сохранение данных в случае сбоя. Это достигается особенностями архитектуры решения (ведение WAL и сохранение данных в ПЗУ).

RocksDB можно рассматривать как умеренно легковесное решение, но с некоторыми особенностями, которые зависят от конкретного использования.

1.3.2.4 Использование RocksDB «из коробки»

RocksDB не является полностью готовым к использованию решением «из коробки», поскольку это библиотека для управления хранилищем, а не полноценная база данных с готовыми высокоуровневыми функциями.

Что доступно

- **Базовая функциональность:** RocksDB предоставляет API для операций с ключами и значениями (вставка, удаление, поиск).
- **Логирование операций:** Поддержка WAL.

Что нужно реализовать самостоятельно

- **Организация логики данных:** RocksDB работает с моделью "ключ-значение и вся логика работы с данными (например, индексация, сложные запросы, схемы данных) должна быть реализована на уровне приложения.
- **Обработка ошибок и восстановление:** Встроенные механизмы RocksDB поддерживают надежность, но необходимо внедрить обработку сбоев, журналирование и восстановление данных на уровне приложения.
- **Дополнительные функции:** RocksDB не имеет встроенных инструментов для работы с высокоуровневыми запросами, такими как SQL. Для этого потребуется либо разработать дополнительный слой, либо использовать существующие решения, например, MyRocks - симбиоз MySQL и RocksDB.
- **Интеграция с языком:** Несмотря на наличие готовых оберток для многих языков, настройка в проекте, особенно при необходимости кастомных конфигураций, остается задачей разработчика.

RocksDB можно начать использовать практически сразу после подключения к проекту для базовых операций, однако его функциональность требует настройки и дополнений. Это делает его подходящим для компетентных разработчиков, готовых настраивать хранилище под свои задачи.

1.3.2.5 Сравнение подходов к конфигурации RocksDB и SQLite

RocksDB можно применять в качестве встроенного хранилища, SQLite по умолчанию является таковым, однако их подходы к конфигурации и применению существенно отличаются, как минимум SQLite - реляционная база данных, но эту

особенность мы проигнорируем. Об основных различиях будет говориться дальше в тексте и более наглядно в таблице 1.9:

- **Гибкость и настройки:** RocksDB предоставляет множество параметров для низкоуровневой конфигурации, таких как политика сжатия, размеры LSM-уровней и управление кэшированием. Это позволяет адаптировать его под конкретные нагрузки, но требует глубокого понимания внутреннего устройства хранилища, а также эмпирического подхода к поиску оптимальных параметров. SQLite, напротив, практически не требует настройки - основные параметры работы устанавливаются автоматически, что делает его проще в использовании.
- **Поддержка SQL:** SQLite предоставляет встроенный SQL-интерфейс. RocksDB же лишен такой «особенности», однако, в случае необходимости, разработчик может реализовать свой собственный интерфейс доступа к данным. Однако для задач, где SQL не является критичной необходимостью, отсутствие встроенного языка запросов не становится ограничением.
- **Простота использования:** SQLite предназначен для случаев, когда нужно минимальное вмешательство в настройку и быстрое начало работы. Его интерфейс "из коробки" обеспечивает комфортное использование даже для разработчиков с небольшим опытом. RocksDB, напротив, ориентирован на использование в сложных системах, где требуется возможность тонкой настройки.
- **Целевая аудитория:** SQLite лучше подходит для малых и средних проектов, где важна простота и скорость внедрения. RocksDB чаще используется в высоконагруженных системах, где низкоуровневый контроль над хранилищем данных является приоритетом.

Итог следующий, подход RocksDB к конфигурации отличается большей сложностью и требовательностью к компетенциям разработчиков. SQLite, в свою очередь, менее оптимальный в сценариях, где необходим «свой» алгоритм работы с данными. Глобально же, если проигнорировать «подкапотную» реализацию этих двух средств, можно сделать следующий примерный вывод - RocksDB сложен для применения в качестве встроенного хранилища, как минимум в вопросе конфигурации и требований к компетенциям разработчика.

Таблица 1.9

Сравнение RocksDB, SQLite и Cassandra

Критерий	RocksDB	SQLite	Cassandra
NoSQL	Да	Нет	Да
LSM-деревья	Да	Нет	Да
Готовая из коробки	Движок, требует интеграции	Готовая база данных	Движок, требует интеграции
Простота настройки	Средняя	Высокая	Низкая
Простота эксплуатации	Средняя	Высокая	Низкая
Требовательность к компетенциям	Средняя	Низкая	Высокая
Универсальность встраивания	Высокая (подходит большинству языков)	Средняя (ограниченные сценарии)	Высокая (API для многих языков)
Универсальность функционала	Средняя (только для ключ-значение)	Низкая	Высокая (распределённые системы)
Встраиваемость	Высокая	Высокая	Низкая

1.3.2.6 Оценка применимости RocksDB в различных языках программирования

RocksDB благодаря своей архитектуре и наличию поддерживаемых обёрток имеет определенную применимость в приложениях на различных языках программирования. Приведём сравнительную таблицу 1.10 работы с RocksDB на разных языках.

Таблица 1.10

Сравнение работы RocksDB с различными языками программирования

Критерий	C++	Java	Golang	Rust
Нативная поддержка	Да, нативный интерфейс	Нет, используется JNI	Нет, используются сторонние обёртки	Нет, используются сторонние обёртки
Провизвод.	Эталонная, полный контроль	Высокая, но возможны накладные расходы	Высокая, зависит от качества обёртки	Высокая, строгий контроль типов
Простота интеграции	Средняя (требуется ручная настройка API)	Высокая (готовая JNI-обёртка)	Средняя (наличие библиотек для Go)	Средняя (требует изучения особенностей)
Отладка	Полный контроль, доступна в C++	Ограниченная, невозможна через средства Java (JDK)	Ограниченная, зависит от реализации	Ограниченная, привязана к особенностям Rust-библиотек.
Управление памятью	Полный контроль	Требуется управление GC и освобождение ресурсов вручную	Управляется Go runtime	Безопасное управление благодаря системе типов
Мануалы	Полная документация	Хорошая, дополнена примерами	Средняя, зависит от сторонних библиотек	Хорошая, но требуется изучение библиотек
Простота настройки	Средняя (требуется глубокое понимание)	Высокая (готовые инструменты)	Высокая (упрощённый доступ через API)	Средняя (необходима настройка через API)

- **C++:** RocksDB написан на C++ и предоставляет нативный интерфейс, что делает его наиболее естественным выбором для использования в приложениях на этом языке. Полный контроль над функциями и возможность отладки обеспечивают эталонную интеграцию.
- **Java:** JNI-обёртка предоставляет удобный интерфейс для работы в среде JVM. Java-разработчики могут использовать RocksDB для высокопроизводительных приложений, но должны учитывать тонкости работы с памятью и накладные расходы JNI, которые, на самом деле, не будут такими уж огромными, это может стать чем-то критичным при большом объеме мелких запросов. Такой опосредованный подход имеет несколько важный недостаток - невозможность отладки посредством средств разработки Java (JDK), этот недостаток актуален вообще для любых обёрток.
- **Golang:** Для Go доступны официальные и сторонние обёртки. Их использование позволяет интеграцию, однако требует дополнительных усилий для настройки и управления ресурсами, учитывая привязку к нативному коду.
- **Rust:** RocksDB также имеет официальные и сторонние библиотеки для интеграции с Rust. Благодаря строгой системе типов и управлению памятью в Rust, использование RocksDB становится безопасным и эффективным, однако работа с интерфейсом требует изучения особенностей библиотек.

RocksDB может быть эффективно использован на всех перечисленных языках, но степень удобства и глубина интеграции зависят от особенностей экосистемы каждого языка.

1.4 Вывод по главе

В данной главе был проведён обзор ключевых моделей баз данных, включая SQL и NoSQL, с акцентом на их особенности, преимущества и недостатки. Рассмотрены концепции ACID и BASE, которые определяют различные подходы к обработке данных, а также структура данных LSM-дерева, являющаяся основой для высокоэффективных систем хранения, оптимизированных под массовую запись и обработку больших объёмов информации.

Отдельное внимание было уделено RocksDB как представителю встраиваемых решений на базе LSM-деревьев. RocksDB - это не универсальное решение «из коробки», а фундаментальный строительный блок для создания специализи-

рованных хранилищ. Его гибкость позволяет разработчикам строить кастомные системы, но одновременно требует глубокого понимания принципов работы и дополнительных усилий по интеграции. Для небольших проектов с ограниченными требованиями использование RocksDB может быть избыточным, в то время как более лёгкие решения, например BadgerDB или SQLite, выглядят предпочтительнее.

Таким образом, в главе были собраны практики и концепции, которые могут служить основой при разработке собственного встроенного легковесного хранилища. Не затрагивались более специализированные аспекты, такие как взаимодействие RocksDB с SSD-накопителями или детальный разбор теоремы CAP на примере Cassandra, чтобы сохранить фокус на прикладных задачах и избежать чрезмерного увеличения объёма работы.

2 ПРОЕКТИРОВАНИЕ ПРОТОТИПА ХРАНИЛИЩА

2.1 Функциональные и нефункциональные требования

Процесс проектирования системы локального хранилища на основе LSM-деревьев^{14,15,16} основывался на ряде определенных требований, отражающих как бизнес-задачи, так и технические ограничения целевой среды.

Во-первых, хранилище должно поддерживать стандартные CRUD-операции: `put(key, value)` для добавления и обновления записей, `get(key)` для извлечения значений по ключу и `delete(key)` для их удаления. При этом необходимо обеспечить «атомарность» батч-флаша - группы изменений, аккумулированных в памяти, - без использования традиционного WAL. В случае внезапного сбоя или отключения питания система должна автоматически удалять незавершённые файлы `SSTable.temp`, проверять контрольные суммы CRC уже сохранённых `SSTable` и исключать повреждённые файлы, после чего загружать метаданные из манифеста и продолжать работу в консистентном состоянии.

Нефункциональные требования диктуются крайне ограниченными ресурсами автономных устройств. Скорость записи в `MemTable` должна обеспечивать не

¹⁴LevelDB documentation. URL: <https://github.com/google/leveldb/blob/main/doc/index.md>

¹⁵Luo C., Carey M. J. *LSM-based storage techniques: a survey* // The VLDB Journal. 2020. Т. 29, № 1. С. 393–418. URL: <https://link.springer.com/article/10.1007/s00778-019-00555-y>

¹⁶O’Neil P. et al. *The log-structured merge-tree (LSM-tree)* // Acta Informatica. 1996. Т. 33. С. 351–385. URL: <https://link.springer.com/article/10.1007/s002360050048>

менее нескольких тысяч операций в секунду на платформе уровня Raspberry Pi, а среднее время ответа на запрос `get(key)` при поддержке Bloom Filter - не более 5 мс, что в целом является довольно средним показателем и в полных системах. Ограничение по оперативной памяти предусмотрено на уровне 250МБ, что диктует использование компактных структур данных и бинарного формата SSTable с минимальными издержками на сериализацию.

Хранилище проектируется как лёгкая Java-библиотека без внешних фреймворков или нативных зависимостей, способная работать на JVM версии 8 и выше. Надёжность обеспечивается за счёт CRC-контроля целостности файлов и атомарного `rename` временных файлов, что гарантирует сохранность при сбое уже зафлашенных батчей.

Наконец, архитектура должна быть конфигурируемой и расширяемой: разработчик может задавать размеры MemTable, количество хеш-функций Bloom Filter, параметры tier-компрессии, а при необходимости - интегрировать собственный механизм компрессии или кэширования. Такие возможности позволяют адаптировать хранилище под широкий спектр сценариев - от IoT-датчиков и носимых устройств до edge-серверов с ограниченным объёмом памяти и I/O.

2.2 Высокоуровневая архитектура решения

В основе хранилища лежит несколько ключевых компонентов, схема которых изображена на рисунке 2.1, каждый из них выполняет свою роль в достижении быстродействия, отказоустойчивости и компактного использования ресурсов.

MemTable. Это основная in-memory структура, куда поступают все операции записи (`put`). В качестве базового контейнера используется отсортированная потокобезопасная карта. MemTable аккумулирует новые пары «ключ–значение» до достижения заданного порога занимаемой памяти, после чего запускается фаза сброса.

Таймер флешинга. Вдобавок к пороговому триггеру по объёму, реализуется фоновый таймер. Он периодически проверяет состояние MemTable и при наличии накопленных данных инициирует сброс даже при отсутствии переполнения. Это гарантирует, что данные не простаивают в памяти слишком долго.

SSTable. Immutable-файлы на диске, представляющие собой последовательность сортированных записей. После сброса MemTable создаётся файл в формате

*.temp, который, пройдя верификацию целостности, атомарно переименовывается в окончательный SSTable. Файлы упорядочиваются и не изменяются после создания.

Compaction Engine. Фоновая подсистема, отвечающая за объединение нескольких SSTable в единый, удаление «мертвых» (удалённых или устаревших) записей и сокращение числа файлов. Реализован упрощённый алгоритм size-tiered компактации.

Bloom Filter.¹⁷ Для каждого SSTable при его создании строится компактная фильтрующая структура. При выполнении операции чтения (get) фильтр позволяет быстро отсеять файлы, где указанный ключ гарантированно отсутствует, тем самым сводя к минимуму ненужные обращения к диску.

Manifest Handler. Компонент, управляющий метаданными хранилища. Он отвечает за поддержание текстового манифеста, в котором фиксируются доступные SSTable и их tiers, а также контрольная сумма целого списка. На старте и после каждой модификации файловой структуры манифест проверяется и обновляется, обеспечивая согласованность между дисковым состоянием и внутренними данными.

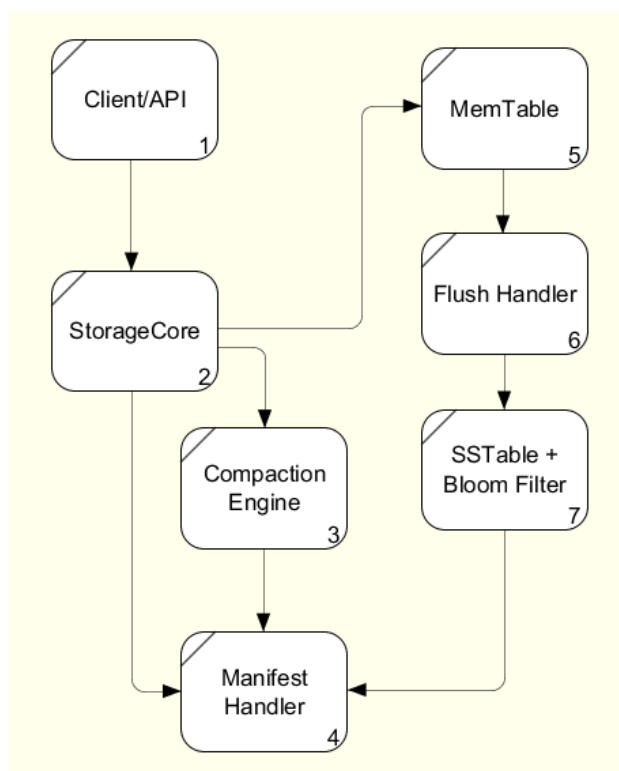


Рис.2.1. Диаграмма компонентов

¹⁷Broder A., Mitzenmacher M. *Network Applications of Bloom Filters: A Survey* // Internet Mathematics. 2002. Т. 1, № 4. С. 485–509. URL: <https://projecteuclid.org/journals/internet-mathematics/volume-1/issue-4/Network-Applications-of-Bloom-Filters-A-Survey/im/1109191032.pdf>

Схема потоков данных

1. Клиент вызывает **put(key, value)**.
2. Пара помещается в MemTable.
3. При переполнении или по таймеру запускается фаза flush. Создаётся snapshot MemTable и сериализуется в файл **SSTable.temp**. После успешного завершения и проверки CRC производится **атомарный rename** в окончательный SSTable.
4. Новые файлы регистрируются в манифесте и становятся доступными для чтения и компактации.

2.3 Интерфейс программной библиотеки

При проектировании API LSM-хранилища были поставлены следующие цели:

1. **Минимальность и ясность.** Интерфейс должен содержать ровно тот набор операций, который нужен подавляющему большинству клиентов: **put, get, delete, flush** и, опционально, **close** для освобождения ресурсов. Никаких лишних методов, которые усложняют понимание и отнимают время на изучение.
2. **Разделение ответственности.** Клиентская часть отвечает только за бизнес-логику (что именно записать и когда прочитать), а все детали - формат SSTable, компактация, отказоустойчивость - скрыты за API. Это упрощает тестирование и развитие библиотеки.
3. **Чёткая семантика ошибок.** Вместо одного «грубого» RuntimeException введены специализированные исключения:
 - **StorageException** для любых проблем ввода-вывода или внутренних сбоев;
 - **InvalidCRC** (или аналог) для ситуации, когда при чтении SSTable нарушена целостность;
 - В будущем можно добавить, например, **CompactionException** или **ConfigurationException**.
4. **Конфигурируемость без усложнения.** Параметры MemTable-size, Bloom-фильтра и thresholds компактации передаются один раз при создании хранилища, после чего API остаётся неизменным.

5. **Явный жизненный цикл.** С момента создания до уничтожения процесса хранилище проходит этапы инициализации (загрузка манифеста, проверка CRC, очистка **.temp**), работы (операции CRUD, фоновые флэши и компактация) и завершения (сохранение состояния, освобождение дескрипторов).

2.4 Формат хранения SSTable

Для каждого SSTable используется компактный бинарный формат, обеспечивающий быстрое чтение и минимальную перегрузку пространства, пример с комментариями файла в таком формате можно увидеть на рисунке 2.2:

1. **CRC32 (8 байт).** В первой части файла записывается контрольная сумма всего последующего содержимого. При открытии SSTable сразу проверяется CRC, что позволяет быстро обнаружить повреждения.
2. **Блок данных.** Содержит упорядоченные записи «ключ–значение». Для каждой пары последовательно записываются: длина ключа (1–2 байта), байтовое представление ключа, длина значения и байты значения.
3. **Блок индексов.** После данных идёт индекс: для каждого ключа записывается его длина, байты ключа и 8-байтовое смещение (offset) в блоке данных. Это позволяет мгновенно переходить к нужной записи без полного сканирования.
4. **Футер (8 байт).** В конце файла хранится указатель (offset) на начало блока индексов, что упрощает его поиск при открытии SSTable.

Выбор бинарного формата продиктован следующими соображениями:

- **Производительность.** Меньше накладных байтов, прямое десериализованное чтение без текстового парсинга.
- **Компактность.** Ключи и значения занимают ровно столько, сколько нужно; нет дополнительных разделителей или escape-символов.
- **Простота навигации.** Чёткая структура «данные → индекс → футер» позволяет за $O(1)$ перейти к индексу и по нему к нужной записи.

1	+-----+-----+	
2	1. CRC32 (8 байт)	
3	+-----+-----+	
4	0x00 00 00 00 AE 4F 12 3C	← CRC32 всего блока данных + индексов + футера
5	+-----+-----+	
6	2. Блок данных	
7	+-----+-----+	
8	0x03	← длина ключа = 3 байта
9	'u' 's' 'r'	← ключ "usr"
10	0x05	← длина значения = 5 байт
11	'A' 'l' 'i' 'c' 'e'	← значение "Alice"
12	0x03	← длина ключа = 3 байта
13	'a' 'g' 'e'	← ключ "age"
14	0x02	← длина значения = 2 байта
15	'3' '0'	← значение "30"
16	0x04	← длина ключа = 4 байта
17	'c' 'i' 't' 'y'	← ключ "city"
18	0x06	← длина значения = 6 байт
19	'L' 'o' 'n' 'd' 'o' 'n'	← значение "London"
20	+-----+-----+	
21	3. Блок индексов	
22	+-----+-----+	
23	0x03	← длина ключа = 3 байта
24	'u' 's' 'r'	← ключ "usr"
25	0x00 00 00 00 00 00 00 08	← offset=8 (начало первой записи в блоке данных)
26	0x03	← длина ключа = 3 байта
27	'a' 'g' 'e'	← ключ "age"
28	0x00 00 00 00 00 00 00 0F	← offset=15 (начало второй записи)
29	0x04	← длина ключа = 4 байта
30	'c' 'i' 't' 'y'	← ключ "city"
31	0x00 00 00 00 00 00 00 17	← offset=23 (начало третьей записи)
32	+-----+-----+	
33	4. Футер (8 байт)	
34	+-----+-----+	
35	0x00 00 00 00 00 00 00 3A	← смещение начала блока индексов = 58
36	+-----+-----+	

Рис.2.2. Визуализация SSTable

2.5 Отказ от WAL и обеспечение отказоустойчивости

Традиционные LSM-хранилища используют механизм Write-Ahead Log (WAL) для гарантии «точно одной» записи каждой операции: все изменения сначала последовательно пишутся в журнал, а затем применяются к основному хранилищу. При сбое систему можно восстановить, «проиграв» WAL, однако этот подход встраиваемым и IoT-решениям даёт сразу несколько серьёзных недостатков:

1. **Двойная запись и нагрузка на И/О.** Каждая операция put сначала записывается в WAL, а затем в MemTable и далее в SSTable. Это удваивает объём дисковой записи и резко увеличивает износ флеш-носителей при высоких нагрузках записи.
2. **Потребление памяти и сложности логики.** WAL требует поддержания буфера для журнала, регулярного сброса на диск с `fsync()` и обеспечения порядка записей. Встроенные устройства часто не могут позволить себе такие сложности и затраты ресурсов.

3. **Ненадёжность в критических моментах.** Даже WAL не даёт абсолютных гарантий: если сбой происходит между записью в журнал и сбросом MemTable на диск, часть операций может оказаться безвозвратно утерянной. Кроме того, повреждённый или неполный WAL при аварии приводит к необходимости удаления или догоняющего восстановления, что усложняет логику и увеличивает время старта.

Вместо WAL в предлагаемом решении реализован механизм атомарного сброса батчей:

- **Глубокая копия MemTable.** Перед записью на диск создаётся «снэпшот» - неизменяемая копия текущего состояния MemTable.
- **Временный файл SSTable.temp.** Снэпшот сериализуется в отдельный файл с расширением **.temp**. В процессе записи поддерживается расчёт **CRC32** по всему содержимому.
- **Атомарный rename.** После успешного завершения записи и верификации CRC временный файл переименовывается в окончательное имя SSTable. Большинство современных файловых систем (POSIX, NTFS, ext4) гарантируют атомарность такой операции: либо файл появляется полностью, либо остаётся старый.
- **Прозрачное восстановление.** При старте хранилища все оставшиеся файлы ***.temp** автоматически удаляются как незавершённые записи. Существующие SSTable-файлы проверяются по CRC; повреждённые удаляются и не участвуют в дальнейшем. После этого система готова к приёму новых запросов без дополнительного «replay» журнала.

Отказ от WAL означает переход от обеспечения атомарной записи каждой операции к обеспечению атомарного сброса батча. Это снижает нагрузку на I/O и упрощает архитектуру, сохраняя гарантии «всё или ничего» на уровне каждого MemTable-флаша. При этом осознанно жертвуются только те изменения, которые не успели попасть в последний батч; в большинстве сценариев автономных и встроенных систем такой компромисс является приемлемым.

2.6 Компактация

Для обеспечения сбалансированной работы LSM-хранилища необходимо регулярно объединять мелкие SSTable-файлы в более крупные. Это позволяет

удалять устаревшие или удалённые записи, снижать фрагментацию и оптимизировать время чтения. В этом пункте будет говориться про Size-Tiered Compaction, выбранную для этого подхода, а также будет проведено сравнение этого механизма с альтернативными, которое можно увидеть в таблицах 2.1 и 2.2.

Таблица 2.1

Сравнение альтернативных стратегий 1/2

Стратегия	Плюсы	Минусы
Size-Tiered (STC)	– Простота реализации – Эффективна при высоком write-throughput – Пакетные merge минимизируют write-amplification	– Возможны пересечения ключевых диапазонов – При чтении нужно проверять несколько файлов
Leveled Compaction	– Минимальные перекрытия файлов – Быстрое чтение одного SSTable	– Частые мелкие merge → высокая I/O нагрузка – Сложность настройки

Таблица 2.2

Сравнение альтернативных стратегий 2/2

Time-Window Compaction	– Оптимизирована для временных рядов – Упрощает работу с «горячими» и «холодными» данными	– Непригодна для произвольных key-value сценариев – Сложности с range-запросами не по времени
Universal Compaction	– Гибкие эвристики выбора файлов – Балансировка write/read	– Сложная логика и высокие накладные расходы CPU

Size-Tiered Compaction (STC):

- SSTable-файлы группируются по размерным диапазонам в «тирах» (Tier 0, Tier 1, ...).
- При накоплении более T файлов в tier выбираются T самых старых и объединяются в один: удаляются устаревшие ключи и записи.
- Новый файл остаётся в том же tier или переходит в следующий при увеличении размера.
- Простая пороговая логика запуска компактации без сложных эвристик.

Многоуровневое хранение (Tiered SSTable)

В данном решении применяется **многоуровневая (tiered) компактация** или упрощённая **size-tiered** стратегия. Схема такого хранилища обозначена на рисунке 2.3:

1. Группировка SSTable по tier

- Файлы размещаются в директориях T0, T1, T2 и т.д. в зависимости от их размера.
- Tier определяется логарифмически:

$$\text{tier} = \left\lfloor \log_2 \left(\frac{\text{fileSize}}{\text{baseSize}} \right) \right\rfloor$$

2. Пороговое объединение

- При накоплении более T файлов в tier выбираются T самых старых и объединяются в новый SSTable.

3. Слияние и продвижение

- Мёртвые записи и старые версии ключей удаляются при merge.
- Новый файл остаётся в текущем tier или продвигается в следующий.

4. Обновление манифеста и Bloom Filter

- Старая группа файлов удаляется, новый SSTable добавляется в манифест.
- Для нового файла строится Bloom Filter для быстрого поиска.

5. Преимущества подхода

- **Минимальная фрагментация**
- **Баланс I/O** за счёт пакетных merge
- **Простота реализации**

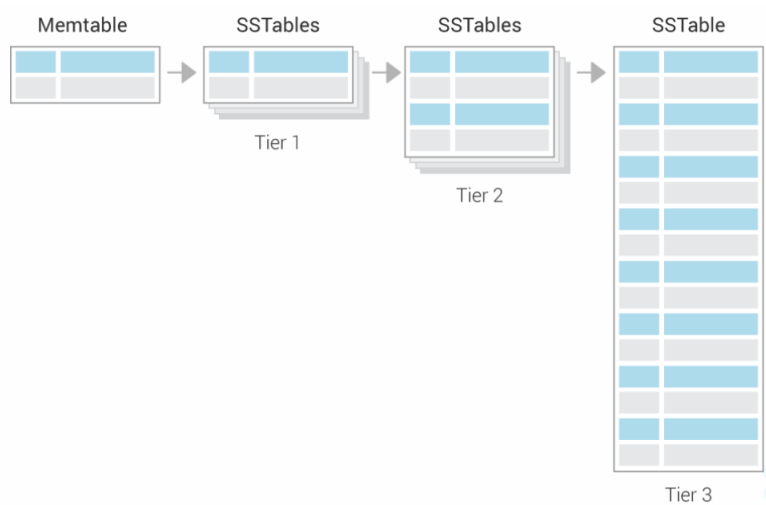


Рис.2.3. Визуализация тиров¹⁸

Выбор в пользу STC: простота, предсказуемость числа файлов и write-amplification, низкая нагрузка на I/O и CPU.

2.7 Bloom-Filter

Одной из проблем при росте числа SSTable-файлов является необходимость обходить множество дисковых структур при выполнении операции `get(key)`. Даже

¹⁸URL: <https://rawdatareaders.hashnode.dev/day-13100>

если каждый отдельный файл содержит небольшой объём данных, суммарная задержка на множество случайных обращений к диску может серьёзно ухудшить производительность.

Для решения этой задачи в архитектуру хранилища встроен **Bloom Filter** - компактный битовый массив, ассоциированный с каждым SSTable. При создании файла в процессе flush мы одновременно строим его Bloom Filter, перебирая набор ключей и устанавливая соответствующие биты под несколькими хеш-функциями. В результате фильтр даёт следующие гарантии: Если Bloom Filter для данного SSTable возвращает **false**, ключа в этом файле точно нет → мы пропускаем обращение к диску. Если он возвращает **true**, ключ возможно присутствует → только тогда мы выполняем фактическое чтение.

Такой механизм позволяет откинуть определенное количество ненужных для чтения файлов без единого seek-запроса, что особенно критично в низкоуровневых устройствах с высокими задержками дискового доступа (но вообще этот подход может быть полезен и не только тут). Небольшая вероятность ложноположительного срабатывания (false positive) на уровне 1–2% полностью компенсируется более быстрой работой системы и остаётся при этом контролируемой за счёт подбора размера битового массива и числа хеш-функций.

2.8 Манифест и метаданные

Для отслеживания набора SSTable-файлов проект предусматривает использование единого текстового манифеста. Манифест содержит строки, в которых записан идентификатор уровня (tier) и имя каждого файла. В начале манифеста хранится контрольная сумма, позволяющая проверить целостность всего списка.

При старте системы происходит проверка манифеста: если файл присутствует и целостность подтверждена, из него загружается информация о доступных SSTable; при обнаружении повреждения или отсутствии манифеста выполняется сканирование всех директорий уровней, после чего создаётся новый манифест.

В процессе работы, после сброса данных из памяти или компактации, манифест обновляется: в него добавляются новые файлы и удаляются устаревшие, затем заново вычисляется контрольная сумма.

Такой подход обеспечивает единый источник правды о состоянии хранилища, позволяет быстро восстанавливать его после сбоев и гарантирует синхронизацию между описанием в манифесте и фактическими файлами на диске.

2.9 Последовательность операций при put и при сбое

Операция put(key, value).

1. Клиент вызывает **put** через API.
2. StorageCore кладёт пару в MemTable.
3. Если размер MemTable превысил порог или таймер флешинга сработал:
 1. Создаётся «снэпшот» MemTable.
 2. Снэпшот сериализуется в SSTable.temp.
 3. По завершении записи и после проверки CRC выполняется атомарный rename → SSTable.dat.
 4. Манифест обновляется: добавляется новый файл с указанием tier и пересчитывается CRC.
 5. MemTable очищается или заменяется на новый.

При внезапном сбое

1. JVM/устройство падает во время или после **put**.
2. При рестарте StorageCore:
 1. Все файлы *.temp удаляются как незавершённые.
 2. Существующие SSTable-файлы проходят CRC-верификацию; повреждённые исключаются.
 3. Манифест пересобирается из валидных файлов.
 4. MemTable инициализируется пустой, таймер флешинга запускается заново.

2.10 Вывод по главе

В ходе проектирования хранилища была выработана компактная, но при этом отказоустойчивая и высокопроизводительная архитектура, адаптированная для ограниченных условий IoT-устройств и встроенных систем.

В качестве буфера операций используется потокобезопасная и сортированная структура MemTable, которая по достижении порогового объёма или по таймеру сбрасывается в дисковый SSTable. Отказ от классического WAL в пользу атомарного механизма флеша: глубокая копия MemTable сериализуется в временный файл, после чего верификация CRC и атомарный ‘rename’ гарантируют «всё или ничего» для каждого батча записей.

Для ускорения чтения каждый SSTable сопровождается компактным Bloom Filter, позволяющим заранее отсеивать файлы без искомого ключа, а структура манифеста с CRC обеспечивает единый источник правды о текущем наборе файлов и поддерживает быстрое восстановление после сбоев.

Компактация реализована по упрощённой size-tiered схеме: файлы группируются в тирах по размеру, а при превышении числа файлов в tier запускается пакетное слияние самых старых SSTable с удалением устаревших записей. Такой подход обеспечивает баланс между скоростью записи, временем чтения и накладными расходами на I/O.

Нефункциональные требования (скорость операций и скромность памяти) были заложены в дизайн, а лёгкий бинарный формат SSTable минимизирует использованное дисковое пространство и повышает производительность сериализации. Предусмотрен расширяемый API без внешних зависимостей, чёткая семантика ошибок и конфигурируемые параметры MemTable-size, Bloom Filter и компакции.

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ РЕШЕНИЯ

3.1 Технологический стек

Реализация LSM-хранилища выполнена на языке Java¹⁹ (минимальная версия - Java 8) без привлечения сторонних фреймворков или нативных библиотек. Сборка и управление зависимостями организованы с помощью Maven.

3.2 Структура репозитория и схема пакетов

Исходный код проекта организован в следующую иерархию:

¹⁹Oracle Java SE 8 Documentation. URL: <https://docs.oracle.com/javase/8/docs/>

```

.
|-- Main.java
|-- exception
|   |-- invalid
|       |-- crc
|           |-- InvalidCRC.java
=-- storage
    |-- bloomfilter
    |   |-- BloomFilter.java
    |-- core
    |   |-- ManifestHandler.java
    |   |-- StorageCore.java
    |-- compengine
    |   |-- CompactationEngine.java
    |-- interfaces
    |   |-- DipLSMStorage.java
    |   |-- MemoryTable.java
    |   |-- SortedStringTable.java
    |-- memtable
    |   |-- MemTable.java
=-- sstable
    |-- SSTable.java
    |-- SSTableMetadata.java

```

Описание пакетов:

- Пакет **storage.interfaces** определяет абстрактные контракты для внешнего API (DipLSMStorage) и базовых операций над MemTable (MemoryTable) и SSTable (SortedStringTable).
- Пакет **storage.memtable** реализует класс MemTable - потокобезопасную in-memory структуру для буферизации записей перед сбросом.
- Пакет **storage.sstable** содержит реализацию на диске: класс SSTable для чтения и записи бинарных файлов, а также SSTableMetadata для хранения метаданных (tier, диапазон ключей, CRC, Bloom Filter).
- Пакет **storage.bloomfilter** включает класс BloomFilter для фильтрации при операциях чтения, для отсеивать файлы без нужного ключа.

- Пакет **storage.compengine** (Compaction Engine) включает `CompactionEngine` для фонового слияния `SSTable`-файлов.
- Пакет **storage.core** объединяет центральный компонент `StorageCore`, управляющий всеми операциями хранилища, и `ManifestHandler`, отвечающий за поддержание и проверку текстового манифеста с информацией о текущих `SSTable`, а также отвечает за начало процесса компактации.

3.3 Описание классов

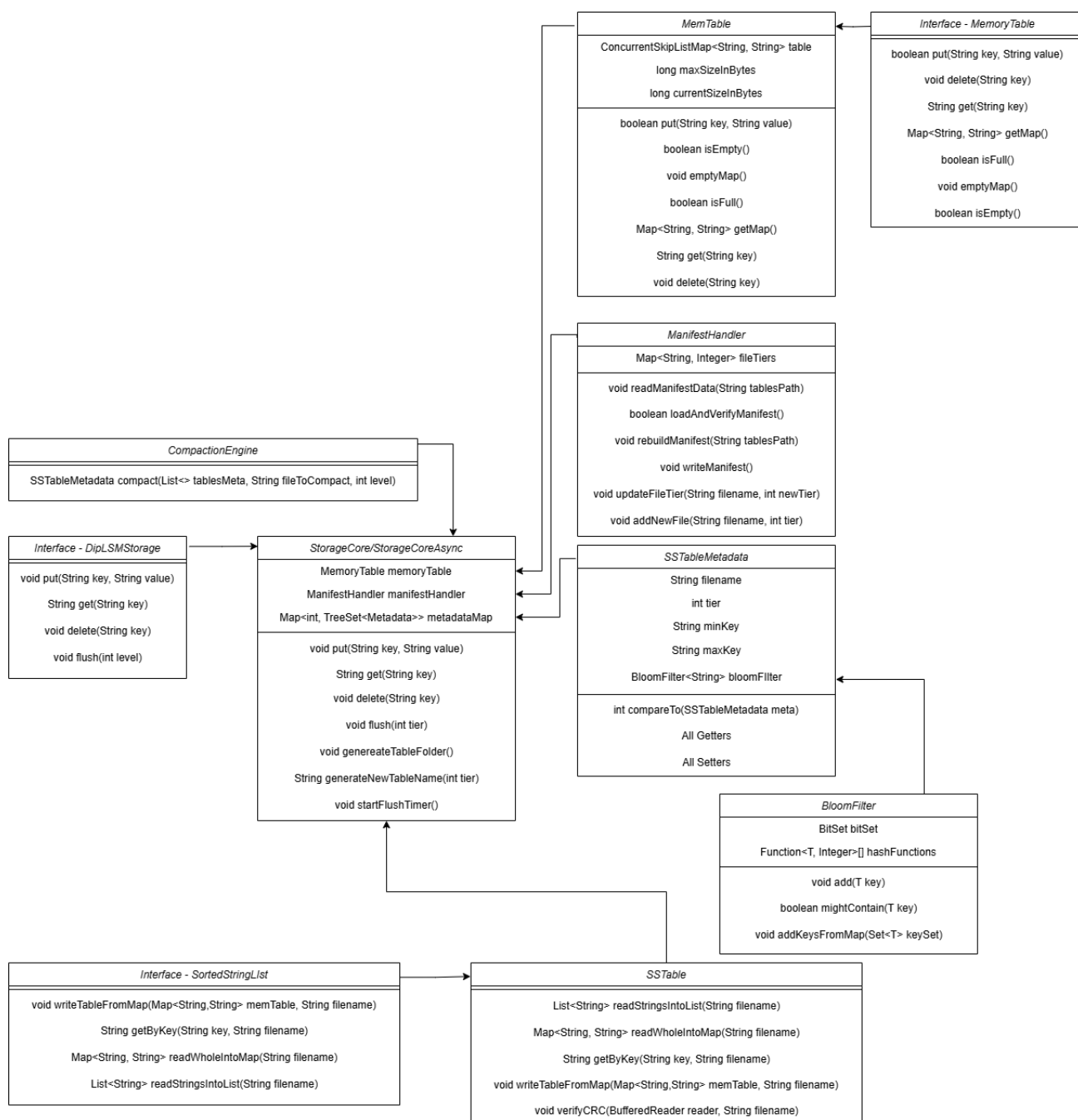


Рис.3.1. Диаграмма классов

Описание объектов системы, изображенных на рисунке 3.1:

1. **DipLSMStorage** - внешний интерфейс, задаёт базовые методы put, get, delete, flush.
2. **StorageCore** - центральный класс, агрегирует все компоненты: MemTable, CompactionEngine, ManifestHandler, метаданные SSTable. Реализует логику маршрутизации запросов и фоновых задач. Начинает процесс компактации
3. **StorageCoreAsync** - модификация **StorageCore**, реализующая процессы флешинга MemTable в SSTable и компактации в отдельном потоке.
4. **MemTable** - потокобезопасная in-memory структура (на основе ConcurrentSkipListMap), минимизирует задержку записи до диска.
5. **SSTable** - класс, отвечающий за сериализацию и десериализацию файлов на диск: включает методы writeTableFromMap, getByKey, readWholeIntoMap, а также verifyCRC.
6. **BloomFilter** - вероятностная структура для быстрого отсева SSTable при чтении, ассоциируется с каждым SSTableMetadata.
7. **ManifestHandler** - управляет текстовым манифестом, обеспечивая согласованность между списком SSTable и их физическим состоянием.
8. **SSTableMetadata** - DTO, содержит имя файла, tier, диапазон ключей (minKey/maxKey) и ссылку на соответствующий BloomFilter. Реализует Comparable для упорядочивания внутри дерева.
9. **CompactionEngine** определяют логику выполнения компактации многоуровневых SSTable.

3.4 Особенности реализации ключевых компонентов

MemTable

В качестве in-memory буфера для операций записи была выбрана ConcurrentSkipListMap. Эта структура обеспечивает потокобезопасное добавление или удаление ключей и поддержание их лексикографически отсортированного состояния. Для контроля потребления памяти каждый вставляемый элемент оценивается по длине ключа и значения в байтах; эти размеры аккумулируются в поле currentSizeInBytes. При превышении заранее заданного предела (maxSizeInBytes) или по срабатыванию фонового таймера создаётся «снэпшот»

текущей карты - копия `ConcurrentSkipListMap`, которая затем сбрасывается в `SSTable`, после чего исходная карта очищается методом `emptyMap()`, а счётчики обнуляются.

SSTable

Для долговременного хранения используется компактный бинарный формат. Первым в файле записывается 8-байтовый CRC32-контроль, защищающий всё последующее содержимое. Далее идёт блок данных: для каждой пары «ключ–значение» записываются длина ключа, его байты, длина значения и его байты. После данных следует блок индексов: для каждого ключа пишется длина, байты ключа и 8-байтовое смещение в блоке данных, что позволяет прямой доступ к нужной записи. Завершает файл футер (8 байт) - указатель на начало блока индексов.

BloomFilter

Реализован на базе `BitSet` и массива хеш-функций (`Function<T, Integer>[]`). При добавлении каждого ключа выполняется k вычислений хешей, а соответствующие позиции в битсете выставляются в 1. Метод `mightContain(T key)` повторно вычисляет те же k хешей и проверяет, все ли биты установлены. Если хотя бы один бит равен 0, ключа в `SSTable` точно нет. Такая реализация даёт контролируемое ложноположительное срабатывание при минимальных затратах памяти.

ManifestHandler

Обменивается с диском данные о текущем наборе `SSTable` через единый текстовый файл - манифест. Первая строка хранит CRC32 всего списка, далее идут строки в формате `T<tier> <filename>`. При инициализации пытается загрузить и проверить существующий файл: если контрольная сумма не совпадает или файл отсутствует, выполняется сканирование папок `tiers` и пересоздание манифеста с перезаписью CRC. После каждого успешного `flush` или компактации манифест обновляется: добавляется новая строка или меняются `tier`-значения, затем CRC пересчитывается и файл перезаписывается целиком.

Compaction Engine

Класс, отвечающий за фоновое объединение SSTable-файлов. Получив на вход карту метаданных (tiers → списки SSTableMetadata) и параметр порога, он находит те tier, где число файлов превысило лимит, выбирает старейшую группу, читает из них все ключи-значения, объединяет их в отсортированную структуру, удаляя устаревшие и удалённые записи, и пишет результат в новый SSTable. Затем старые файлы удаляются, metadata обновляются, а манифест актуализируется. Компактация может запуститься после очередного флешинга.

3.5 Поддержка расширяемости

Архитектура хранилища спроектирована с учётом будущей доработки и интеграции сторонних реализаций. Для Bloom Filter и Compaction Engine предусмотрен механизм SPI (Service Provider Interface): в конфигурации можно указать полностью заменяющий класс, реализующий интерфейс BloomFilter<T> или CompactionEngine. Это позволяет, например, переключиться на Counting Bloom Filter, Cuckoo Filter или продвинутый leveled компактор без изменения клиентского кода.

Кроме того, формат SSTable и способ сериализации легко заменяемы: вместо бинарного формата может быть подключена альтернативная реализация, реализующая тот же контракт, например сохранение файлов в формате "plain-text" - без индексов и бинарностей, но в более когнитивно понятном виде - что позволит адаптировать хранилище под другие требования к совместимости или аналитике. Такая модульность делает библиотеку гибкой и готовой к расширению в рамках различных сценариев использования.

3.6 Потенциал развития

Разработка **инструментов миграции и бэкапа** - одно из ключевых направлений дальнейшей работы. CLI-утилиты для экспорта и импорта SSTable обеспечат возможность перемещения данных между инстансами хранилища. Механизм горячих бэкапов (copy-on-write) позволит создавать снимки текущего состояния без прерывания записи. Это упростит процесс обновления системы, резервного копирования и восстановления предыдущих версий данных при сбоях.

Оптимизация кэша блоков SSTable - ещё одно важное направление. Анализ реальных сценариев работы показал, что многие запросы обращаются к одним и тем же диапазонам данных. Внедрение кэша популярных блоков в оперативной памяти с LRU-стратегией замещения и динамической настройкой размера под доступный объём RAM позволит сократить количество обращений к диску. Это особенно актуально для платформ с медленными флеш-носителями, где даже случайный seek может занимать значительное время.

Динамическая настройка компактации также является важной задачей. В IoT-системах и микросервисах интенсивность операций часто меняется, что требует адаптации параметров size-tiered компактации на основе анализа метрик throughput и задержек в реальном времени. При необходимости система должна автоматически переключаться на leveled стратегию для минимизации латентности чтения.

3.7 Вывод по главе

В этой главе было показано, как спроектированная архитектура превратилась в полноценный прототип в виде Java-библиотеки. Основной акцент был сделан на:

Аскетичность решения: отказ от WAL реализован через атомарный сброс батчей (snapshot → временный файл → переименование), что обеспечивает «всё или ничего» без лишних I/O-нагрузок и упрощает восстановление после сбоя.

Эффективности записи и чтения: MemTable на базе ConcurrentSkipListMap хранить и автоматически сортирует записи после каждого нового put'а, а встроенные в SSTable Bloom-фильтры помогают отсеивать ненужные файлы при поиске ключа, уменьшая нагрузку на I/O.

Компактном бинарном формате SSTable: строгое разделение на блок данных, индекс и футер с CRC даёт предсказуемое время доступа и минимальную фрагментацию, а также поддержку целостности данных.

Простом в расширении и настройке: через SPI-модель можно заменить алгоритмы фильтрации и компактации без изменений в пользовательском коде, а параметры MemTable-размера, порогов компактации и характеристик Bloom-фильтра настраиваются при инициализации.

Велика вероятность, что дальнейшая разработка будет направлена на именно на добавление инструментов бэкапа и миграции, реализацию кэша и на адап-

тивные механизмы компактации. Эти улучшения позволят довести решение до промышленного уровня и расширить круг его применения.

4 ТЕСТИРОВАНИЕ И АПРОБАЦИЯ РЕАЛИЗОВАННОГО РЕШЕНИЯ

4.1 Введение в автономные системы и ограниченные ресурсы

В современном мире всё большее распространение получают автономные и edge-устройства, выполняющие разнообразные задачи без постоянного подключения к центральным серверам. К таким системам относятся промышленные контроллеры, бытовые IoT-датчики, автономные робототехнические комплексы, носимая электроника и многие другие устройства.

Во-первых, такие системы предъявляют жёсткие требования к аппаратным ресурсам. Процессор может быть энергоэффективным, но слабым по вычислительной мощности; объём оперативной памяти зачастую не превышает нескольких сотен мегабайт, а место на флеш-накопителе тоже ограничено. Кроме того, дисковая подсистема на многих устройствах представлена eMMC или медленными SD-картами с невысокой пропускной способностью и ограниченным ресурсом перезаписи. Важно, чтобы на этих платформах вся логика записи и чтения работала предсказуемо и не приводила к резким ”провалам” в производительности или чрезмерному износу носителя.

Во-вторых, автономные решения часто работают без гарантированного питания: в них могут применяться батареи или энергосберегающие режимы, при которых устройство мгновенно переходит в спящий режим. В таких условиях недопустима потеря критичных данных из-за внезапного отключения питания. Любая задержка, связанная с записью журналов (WAL) или долгой синхронизацией данных на диск, может стать причиной потери последних записей.

Наконец, цель тестирования, описанного в этой главе, заключается в том, чтобы показать: предложенный прототип лёгкого LSM-хранилища на Java может сохранять свойства производительности и отказоустойчивости даже на слабых и нестабильных платформах. Была проверена корректность операций на уровне юнит-тестов, измерены задержки записи и чтения при реальной нагрузке, а также замерена динамика роста потребления памяти. Всё это проводилось с ориентацией на среду edge-устройств: минимальная зависимость от сторонних библиотек,

отсутствие тяжёлых фоновых процессов и использование только встроенных механизмов JVM.

4.2 Среда тестирования и инструменты

Для теста разработанного LSM-хранилища была организована стандартная ПК-платформа, с целью установления максимально эффективных метрик решения. Также планировалось перенести эти же тесты в контейнеры Docker с ограниченными ресурсами, чтобы симулировать работу на маломощных устройствах, однако первые замеры проводились на настольном компьютере без жёстких ограничений.

4.2.1 Аппаратная платформа

Тесты выполнялись на относительно мощном настольном компьютере - процессор AMD Ryzen 5 4500U (6 ядер, 2.3 ГГц), 16 GB SODIMM DDR4 (2667 MT/s) и NVMe-SSD WD Blue SN570. Такая конфигурация избыточна для автономных и edge-устройств, однако она позволила добиться стабильных и воспроизводимых результатов: без беднейшего железа мы получили эталон производительности. Более того, сильная платформа дала возможность напрямую сравнить собственную Java-реализацию LSM-хранилища с промышленными аналогами (MapDB, BadgerDB и RocksDB) в одинаковых условиях, выявив узкие места и преимущества каждого подхода.

4.2.2 Программная платформа

В качестве целевых runtime-окружений для сравнения были выбраны три языка и их распространённые версии:

- **Java 8** - поскольку на многих встраиваемых системах по-прежнему используются старые JVM.
- **Golang 1.24.0** - современная, но при этом компактная среда выполнения, часто применяемая в микросервисах. В качестве хранилища использовалось BadgerDB
- **C++17** - с библиотекой RocksDB в качестве "золотого стандарта" LSM-хранилищ на нативном коде.

Все первоначальные эксперименты (модульные и нагрузочные тесты) проводились в среде Windows. При этом тестовые программы написаны так, чтобы не зависеть от специфики ОС: в Java используется только стандартная библиотека, в Go - только встроенные пакеты, в C++ - STL и API RocksDB. В дальнейшем аналогичные тесты будут перенесены в контейнеры Docker для моделирования сред с ограниченным объёмом памяти и вычислительной мощности.

Параметры Java-LSM-хранилища на время всех тестов:

- Максимальный размер Memory Table - 1 MB.
- Максимальное количество файлов на всех уровнях, кроме последнего - 5.

4.3 Unit-тестирование как проверка корректности и устойчивости

Важным этапом валидации любого программного решения являются модульные (unit) тесты, которые позволяют убедиться в том, что базовые операции работают правильно и сохраняют целостность даже в граничных ситуациях. В нашем Java-прототипе LSM-хранилища без WAL-журнала ключевыми компонентами для проверки стали: классы **SSTable** (запись, чтение, удаление), механизм переименования временных файлов и CRC-контроль, а также менеджер манифеста - **ManifestHandler**.

4.3.1 Тест-кейсы для операций put/get/delete

Для проверки функционала **put**, **get** и **delete** в **StorageCore** написаны тесты, которые имитируют поведение приложений и проверяют корректность выполняемых действий:

- Сохраняют несколько ключ-значение в MemTable, заставляя MemTable "переполниться" и вызвать flush.
- Проверяют, что после **put** значение доступно как в памяти, так и на диске.
- Выполняют последовательность удаления - **delete(key)** гарантированно удаляет ключ из MemTable и из вновь создаваемых SSTable.
- В сценариях с повторным **put** на тот же ключ убеждаются, что старое значение перезаписывается.

4.3.2 Тестирование CRC-проверки и механизма переименования

Класс **SSTable** обёрнут в два уровня проверок:

1. При записи сначала создаётся временный файл с расширением **.temp**; только после полного успешного формирования контента он переименовывается в окончательный **.sst**-файл. Тесты искусственно выбрасывали исключение между этими этапами, а затем проверяли, что:
 - Файл **.temp** либо удалён, либо остаётся отдельно, но **.sst** всегда содержит только корректно сформированные данные.
 - При инициализации чтения игнорируются незаконченные **.temp**-файлы.
2. В тестах CRC-проверки формировались случаи, когда содержимое **SSTable** намеренно портили (меняли байт после записи CRC). Метод **getByKey** и чтение всей таблицы должны были обнаружить рассинхронизацию контрольной суммы и бросить **InvalidCRC**.

4.3.3 Тесты *ManifestHandler*

ManifestHandler ответственен за поддержание файла **MANIFEST**, который хранит соответствие имён **SSTable**-файлов их уровням. Тесты проверяют:

- Правильность чтения манифеста при старте: если **MANIFEST** отсутствует, он реконструируется сканированием директорий **T0...TN**.
- Корректная реакция на "порчу" **MANIFEST** (изменена или удалена строка CRC) - в этом случае обнуляется манифест и восстанавливается из файловой системы.
- Операции **addNewFile**, **updateFileTier** и **postCompactionRebuild** добавляют и удаляют записи в манифесте, а при последующем перезапуске приложения файл **MANIFEST** остаётся в целостном состоянии.

Результаты unit-тестов

Тестируемые модули показывают успешное прохождение всех описанных и воспроизведенных тест-кейсов. Никаких проблем с функционалом или логикой не обнаружено.

4.4 Нагрузочное тестирование: пропускная способность и потребление ресурсов

В рамках нагрузочного тестирования было проверено, насколько полученная реализация LSM-хранилища справляется с интенсивной записью и чтением при одновременном контроле потребления памяти.

Использовались два сценария на Java:

- **MixedTest** - попеременные операции put и get, имитирующие поведение микросервиса на edge-устройстве;
- **PutOnlyTest** - сплошная запись без чтения, с регулярным снятием показателей памяти.

Для Go и C++ был запущен лишь первый сценарий - MixedTest. И для сопоставимости он был запущен в одних и тех же таймингах: 1 минута и 10 минут. Хотя для Go и RocksDB "прогрев" JVM не требуется, что очевидно, это нужно для обеспечения сравнительной однородности методики.

4.4.1 MixedTest (Java 8)

Для Java 8 сценарий MixedTest продемонстрировал чрезвычайно высокую пропускную способность. За первую минуту было выполнено почти 26,6 млн операций (13,3 млн put и 13,3 млн get) со средними задержками **put = 0,00106 ms** и **get = 0,00247 ms**, а за десятиминутный прогон - 238,7 млн операций, с **put = 0,00116 ms** и **get = 0,00283 ms**. Ошибок согласованности (**mismatches**) не было. Результаты MixedTest в сравнении приведены на таблицах 4.1 и 4.2.

4.4.2 PutOnlyTest (Java 8)

Режим "только put" показал, как меняется память при интенсивной записи, графики изменения занимаемой памяти по прошествии времени изображены на рисунке 4.1. За первую минуту было совершено $\approx 43,6$ млн **put** со средней задержкой **0,00073 ms**, а потребление памяти выросло с 5 MB до 248 MB. Регулярные замеры каждые 1 млн операций (примерно раз в секунду) выявили, что footprint колебался от 48 MB до 490 MB, что объясняется накоплением метаданных SSTable и overhead-ом на вывод в консоль. При отключении печати онлайн-метрик потребление заметно снизилось - это подчёркивает, что даже логирование "в большую" может серьёзно влиять на RAM в условиях edge-устройств.

За десять минут PutOnlyTest аналогично совершил сотни миллионов **put** и показал схожую латентность, а пиковые потребления иногда приближались к одному гигабайту - ясно, что без оптимизации метаданных под более скромные объемы оперативной памяти, свойственной встраиваемым и автономным устройствам, такое решение работать не сможет. Окончательно обозначить или опровергнуть необходимость оптимизаций можно будет после тестов в искусственной ограниченной среде. О них будет говориться немного позже.

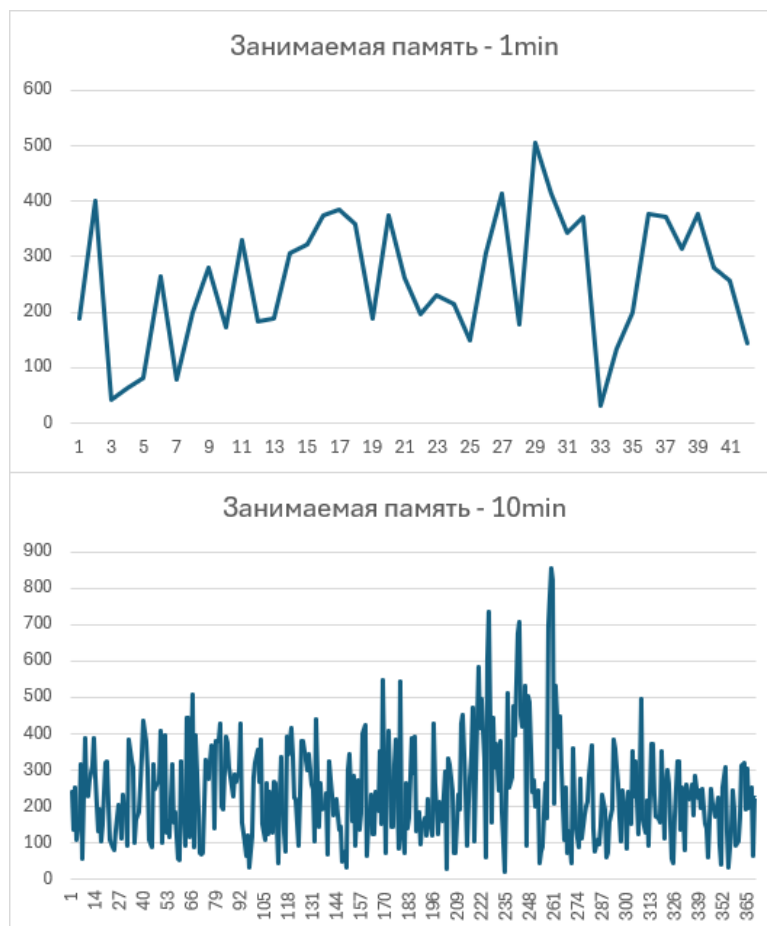


Рис.4.1. График использования памяти в "мощной" среде

4.4.3 Go + BadgerDB

В той же mixed-схеме на Go 1.21 BadgerDB за первую минуту обработал 4,1 млн операций (2,06 млн **put** + 2,06 млн **get**) с **put** = 0,014 ms и **get** = 0,012 ms, а за 10 минут - 29,9 млн операций с **put** = 0,015 ms и **get** = 0,021 ms. Сходимость результатов за разные длительности свидетельствует о практически отсутствующем "прогреве" в Go-рантайме.

4.4.4 C++ + RocksDB

В аналогичных mixed-тестах на C++ 17 с RocksDB за первую минуту получилось 4,66 млн операций (2,33 млн **put** и 2,33 млн **get**) с **put** = **0,0163 ms** и **get** = **0,0074 ms**, а за 10 минут - 48,8 млн операций с **put** = **0,0155 ms** и **get** = **0,0071 ms**.

4.4.5 Выводы по нагрузочному тестированию

Нагрузочное тестирование выявило следующие ключевые факты:

1. **Java-LSM** превосходит Go и C++ в скорости **put** и не проигрывает им в скорости **get**.
2. **Go + BadgerDB** сбалансирован по **put/get** (15/15 μ s).
3. **C++ + RocksDB** даёт лучшие **get** (7 μ s), но **put** немного медленнее Java.

Для полноценных edge-приложений можно предложить следующие улучшения:

- Нужно переработать хранение метаданных в Java-прототипе (sparse-индексы, уменьшить tierThreshold).
- Уменьшить частоту flush (большой MemTable) и/или упростить manifest, чтобы не накапливать сотни мелких SSTable.
- Применить ту же нагруженность и мониторинг в Docker-контейнерах с ограниченными CPU/RAM, что будет описано в дальнейшем.

Сводные таблицы

Таблица 4.1

Средние латентности операций за минутное тестирование (MixedTest)

Система	Put, ms	Get, ms
Java + LSM	0,001056	0,002468
Go + BadgerDB	0,014187	0,012114
C++ + RocksDB	0,016303	0,007421

Таблица 4.2

Средние латентности операций за десятиминутное тестирование (MixedTest)

Система	Put, ms	Get, ms
Java + LSM	0,001158	0,002834
Go + BadgerDB	0,015432	0,021383
C++ + RocksDB	0,015484	0,007086

4.5 Тестирование в ограниченной среде

В предыдущих главах была исследована производительность разработанного хранилища на ”мощном” настольном стенде. Однако реальные автономные или edge-устройства зачастую располагают совсем другими ресурсами: скромным CPU, ограниченным объёмом памяти и низкоскоростным или изношенным накопителем. Чтобы максимально приблизить испытания к таким условиям, нагрузочные тесты были перенесены внутрь Docker-контейнера, строго ограничив его ресурсы: не более **0,5 ядра процессора** и **512 MB оперативной памяти**.

Цель такого эксперимента - выяснить, насколько корректно JVM и само хранилище ”самооптимизируются” под заданные ограничения, не теряя при этом ключевых характеристик: низкой латентности операций и отсутствия утечек памяти.

4.5.1 Конфигурация тестового стенда

1. **Аппаратная/виртуальная среда.** Все контейнеры запускались на стандартной системе (AMD Ryzen 5 4500U, 16 GB RAM, NVMe-SSD), под управлением Docker. Этот выбор платформы обусловлен возможностью быстро конфигурировать и перезапускать контейнеры с разными ограничениями.
2. **Docker Compose.** Файл **docker-compose.yml** настроен так, чтобы контейнер получал:
 - **mem_limit: 512M** - общий объём оперативной памяти не более 512 MB;
 - **cpus: 0.5** - не более половины логического CPU;
 - **JAVA_OPTS = -Xmx512m** - максимальный размер heap JVM, равный 512 MB.

3. **Сценарии запуска.** Внутри контейнера последовательно выполнялись те же тесты, что и на ”мощном” стенде:

- **MixedLoadTest** - комбинация put+get с проверкой консистентности (длительность 1 минута и 10 минут).
- **PutOnlyLoadTest** - интенсивная запись, измерение латентности и снимки потребления памяти через равные интервалы (1 минута и 10 минут), графики изменения памяти изображены на рисунке 4.2.

4.5.2 Анализ полученных результатов

MixedTest

За 1 минуту:

- Всего операций: 8 256 436 (puts = 4 128 218, gets = 4 128 218)
- Средняя латентность:
 - put - 0,00445 ms ($\approx 4,45 \mu\text{s}$)
 - get - 0,00356 ms ($\approx 3,56 \mu\text{s}$)

За 10 минут:

- Всего операций: 39 126 452 (puts = 19 563 226, gets = 19 563 226)
- Средняя латентность:
 - put - 0,01162 ms ($\approx 11,62 \mu\text{s}$)
 - get - 0,00399 ms ($\approx 3,99 \mu\text{s}$)

PutOnly

За 1 минуту:

- Всего put: 7 964 950
- Средняя латентность put - 0,00354 ms
- Память: выросла с 2 МВ до 4 МВ (в конце теста), пики в: 21 МВ, 23 МВ, 18 МВ, ...

За 10 минут:

- Всего put: 90 329 702
- Средняя латентность put - 0,00329 ms
- Память: выросла с 2 МВ до 14 МВ (в конце теста), пики: 24 МВ, 23 МВ, 21 МВ, ...

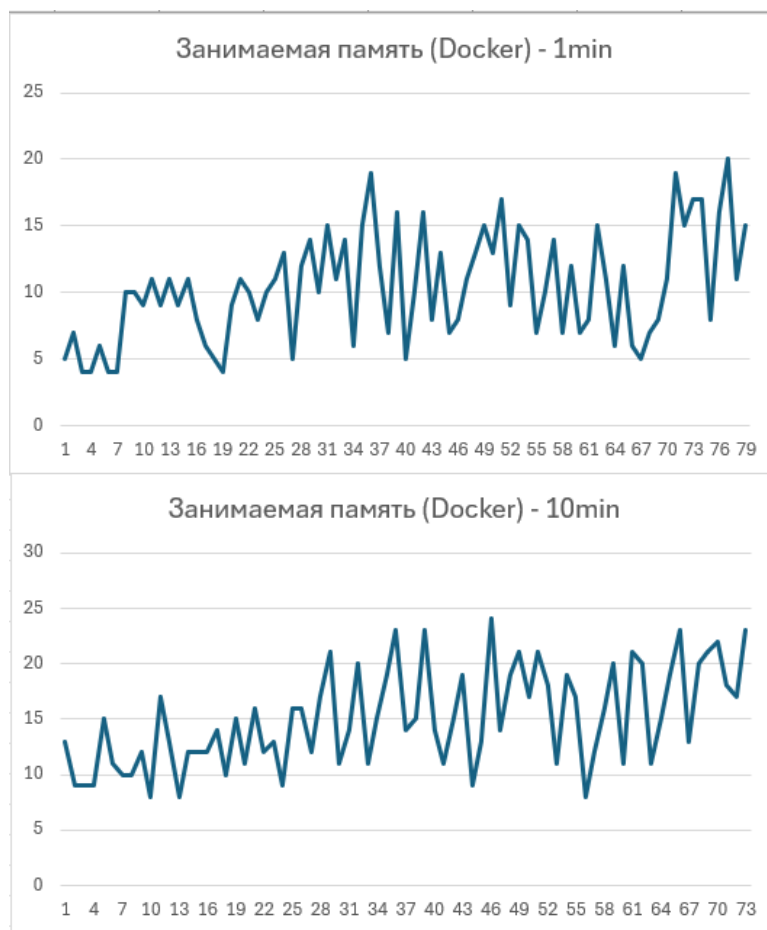


Рис.4.2. График использования памяти в ограниченной среде

4.5.3 Сравнение с "мощной" средой

При переносе нагрузочных тестов из локального высокопроизводительного стенда в ограниченный Docker-контейнер сразу проявились две ключевые закономерности.

Задержка операций

- На "мощном" стенде Java-LSM показало среднюю задержку put порядка 0,001 ms и get 0,002-0,003 ms, более подробные данные записаны в таблице 4.4.
- В контейнере эти значения выросли, но остались в микросекундном диапазоне, что можно увидеть в таблице 4.3:
 - Для MixedTest на 1 минуту put = 0,00445 ms, get = 0,00356 ms;
 - На 10 минут put = 0,01162 ms, get = 0,00399 ms. Увеличение latency put в долгом прогоне связано с фоновой активностью GC и компактацией, однако даже пик 12 μ s остаётся приемлемым .

Таблица 4.3

Метрики латентности и количества операций для MixedTest в Docker

Интервал	Всего оп.	Puts	Gets	Put, ms	Get, ms
1 минута	8 256 436	4 128 218	4 128 218	4,45 µs	3,56 µs
10 минут	39 126 452	19 563 226	19 563 226	11,62 µs	3,99 µs

Таблица 4.4

Метрики латентности и количества операций для MixedTest в ”мощной” среде

Интервал	Всего оп.	Puts	Gets	Put, ms	Get, ms
1 минута	26 599 884	13 299 942	13 299 942	1,05 µs	2,48 µs
10 минут	238 704 786	119 352 393	119 352 393	1,15 µs	2,83 µs

Потребление памяти

- На локальном стенде в режиме PutOnlyTest footprint достигал сотен мегабайт, а порой и гигабайта, из-за накопления метаданных SSTable и агрессивного логирования.
- В контейнере, несмотря на десятки миллионов put, пиковое потребление не превышало 24 МВ, а в среднем удерживалось в пределах 10-20 МВ.

Таблица 4.5

Сравнение потребления памяти PutOnlyTest на ”мощном” и ”маломощном” стендах

Стенд	1 минута		10 минут	
	Сред. (МВ)	Пик (МВ)	Сред. (МВ)	Пик (МВ)
Мощный (локальный)	258	504	239	855
Маломощный (Docker)	10	20	15	24

При жёстких ресурсных лимитах среды (CPU: 0.5, RAM: 512 МВ) разработанное хранилище слабо теряет в производительности и демонстрирует скромный footprint. Метрики PutOnly теста можно увидеть в таблице 4.5

4.5.4 Анализ адаптации JVM

Важный вывод тестов в контейнере заключается в том, что JVM ”самооптимизируется” под заданные ограничения. Во время конфигурации среды был задан параметр, ограничивающий максимальный размер кучи, -Xmx512m,

и JVM чётко соблюдает это ограничение, в отличие от поведения на локальной машине, где default-настройки могли "расширять" кучу до нескромных значений.

В контейнере сборщик мусора (G1 по умолчанию в Java 8+) работает более агрессивно: при достижении низких свободных порогов он быстрее освобождает неиспользуемые объекты, что и объясняет небольшой размер кучи.

Отсутствие частых log-выводов позволило исключить накопление внутренних буферов `PrintStream`, что дополнительно снизило footprint.

В сумме контейнерные ограничения стимулируют JVM к компактному и предсказуемому использованию памяти, а встроенная система GC адаптируется к скромным условиям тестового стенда.

4.6 Выводы по главе

В ходе многопланового тестирования были подтверждены достижения поставленных целей: обеспечить корректность и отказоустойчивость базовых операций в Java-реализации LSM-хранилища, измерить его реальную производительность и оценить потребление памяти в режиме интенсивной записи и смешанных нагрузок как в стандартных ПК системах, так и в ограниченных средах. Юнит-тесты показали безошибочное прохождение ключевых сценариев, а нагрузочные бенчмарки продемонстрировали средние латентности записи и чтения, сопоставимые с промышленными аналогами.

Тесты в контейнере с жёсткими ограничениями (0.5 CPU, 512 MB RAM, -Xmx512m) показали, что предложенное Java-LSM-хранилище сохраняет низкий footprint (не более 20 MB в режиме записи) и высокую пропускную способность (латентности $< 12 \mu s$). При этом прежние опасения о росте памяти до сотен мегабайт оказались напрасными: JVM эффективно очищает неиспользуемые объекты, а отключение частого логирования ещё сильнее снижает объем кучи.

Общий итог: без дополнительной доработки Java-LSM-хранилище вполне готово к практическому применению в автономных и edge-системах с ограниченными ресурсами. Призовое выступление на конференции "Современные технологии в теории и практике программирования" подтвердило актуальность подхода.

ЗАКЛЮЧЕНИЕ

В ходе работы были всесторонне изучены LSM-деревья как механизм для задач хранения данных с высокой интенсивностью записи: их способность собирать записи в памяти и сбрасывать их пачками на диск действительно снижает число случайных I/O-операций и повышает пропускную способность системы. Классический подход с Write-Ahead Log показал свою надёжность в крупных СУБД, но оказался избыточным и ресурсоёмким для встраиваемых и IoT-приложений.

На базе этого анализа была спроектирована и реализована нативная Java-библиотека, в которой отказ от WAL заменён атомарным сбросом "снэпшота" MemTable → временный SSTable → переименование в постоянный файл с CRC-проверкой. В качестве буфера используется потокобезопасная структура ConcurrentSkipListMap, каждая SSTable сохраняется в компактном бинарном формате (data-block → index-block → footer), а при чтении ключей Bloom Filter отсекает ненужные файлы без лишних дисковых операций над ними. Создан фоновый механизм size-tiered компактации, а манифест с метаданной о хранимых файлах SSTable позволяет корректно продолжить работу системы после запланированных отключений или сбоев. Все компоненты - от MemTable и SSTable до CompactionEngine и ManifestHandler - реализованы в чистой Java без сторонних зависимостей, что гарантирует лёгкую интеграцию и минимальный «отпечаток» встраиваемой системы.

Перспективы дальнейшей работы включают разработку утилит для миграции и горячего бэкапа, кэширования «горячих» блоков SSTable и динамической адаптации параметров компактации в зависимости от реальной нагрузки. Полученный прототип демонстрирует жизнеспособность подхода и закладывает надёжную основу хранилища.

Ещё немало важным этапом разработки является сравнение этого решения с популярными альтернативными решениями, при том и на других языках программирования, на стендах, эмулирующих ограниченную среду. Доступным и "нежелезным" решением является Docker, который позволяет задать параметры тестового стенда - максимальный размер ОЗУ, ограничение вычислительных ресурсов и т.д. и т.п.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Клеппман М. *Высоконагруженные приложения. Программирование, масштабирование, поддержка*. Москва: ДМК Пресс, 2022. – 576 с.
2. Apache Cassandra Documentation. URL: <https://cassandra.apache.org/doc/latest/>. Дата обращения: 10.01.25
3. RocksDB Wiki. URL: <https://github.com/facebook/rocksdb/wiki>. Дата обращения: 10.01.25
4. O’Neil P. et al. *The log-structured merge-tree (LSM-tree)* // Acta Informatica. 1996. Т. 33. С. 351–385. URL: <https://link.springer.com/article/10.1007/s002360050048>. Дата обращения: 10.01.25
5. LevelDB documentation. URL: <https://github.com/google/leveldb/blob/main/doc/index.md>. Дата обращения: 15.04.25
6. Dong S. et al. *Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications* // ACM Transactions on Storage (TOS). 2021. Т. 17, № 4. С. 1–32. URL: <https://dl.acm.org/doi/full/10.1145/3483840>. Дата обращения: 10.01.25
7. Luo C., Carey M. J. *LSM-based storage techniques: a survey* // The VLDB Journal. 2020. Т. 29, № 1. С. 393–418. URL: <https://link.springer.com/article/10.1007/s00778-019-00555-y>. Дата обращения: 10.01.25
8. Binani S., Gutti A., Upadhyay S. *SQL vs. NoSQL vs. NewSQL — a comparative study* // Database. 2016. Т. 6, № 1. С. 1–4. URL: <https://www.caeaccess.org/archives/volume6/number1/binani-2016-cae-652418.pdf>. Дата обращения: 13.01.25
9. Sharma V., Dave M. *SQL and NoSQL databases* // International Journal of Advanced Research in Computer Science and Software Engineering. 2012. Т. 2, № 8. URL: https://www.researchgate.net/profile/Meenu-Dave/publication/303856633_SQL_and_NoSQL_Databases/links/5758557f08ae9a9c954a7573/SQL-and-NoSQL-Databases.pdf. Дата обращения: 13.01.25
10. Sethi S., Panda S. *SQL or NoSQL—Practical Aspect and Rational behind Choosing Data Stores* // Journal of Computer and Communications. 2024. Т. 12, № 8. С. 155–174. URL: <https://www.scirp.org/journal/paperinformation?paperid=135134>. Дата обращения: 13.01.25

11. MapDB Documentation. URL: <https://github.com/jankotek/mapdb/wiki>. Дата обращения: 25.02.25
12. Broder A., Mitzenmacher M. *Network Applications of Bloom Filters: A Survey* // Internet Mathematics. 2002. Т. 1, № 4. С. 485–509. URL: <https://projecteuclid.org/journals/internet-mathematics/volume-1/issue-4/Network-Applications-of-Bloom-Filters-A-Survey/im/1109191032.pdf>. Дата обращения: 20.03.25
13. Oracle Java SE 8 Documentation. URL: <https://docs.oracle.com/javase/8/docs/>. Дата обращения: 10.02.25
14. JUnit 5 User Guide. URL: <https://junit.org/junit5/docs/current/user-guide/>. Дата обращения: 09.05.25
15. Go Programming Language Specification (Go 1.21). URL: <https://go.dev/ref/spec>. Дата обращения: 10.05.25
16. BadgerDB Documentation. URL: <https://pkg.go.dev/github.com/dgraph-io/badger/v4>. Дата обращения: 10.05.25
17. Docker Documentation: Resource constraints. URL: https://docs.docker.com/config/containers/resource_constraints/. Дата обращения: 13.05.25
18. Виниченко Д.Н., Андрианова Е.Е. *Применение LSM-деревьев для локальных NoSQL-хранилищ в автономных системах* // Современные технологии в теории и практике программирования: сб. материалов науч.-практ. конф. студентов, аспирантов и молодых учёных, Санкт-Петербург, 23–24 апреля 2025 года. — СПб.: Политех-Пресс, 2025. — С. 28–30. — URL: <https://www.elibrary.ru/item.asp?id=81437269&pff=1> Дата обращения: 24.04.25

Приложение 1

Код приложения

Листинг П1.1

StorageCore.java - ядро сервиса

```

package ru.choomandco.diplsm.storage.core;

import ru.choomandco.diplsm.storage.compengine.
    CompactationEngine;
5 import ru.choomandco.diplsm.storage.interfaces.CompEngine;
import ru.choomandco.diplsm.storage.interfaces.DipLSMStorage;
import ru.choomandco.diplsm.storage.interfaces.MemoryTable;
import ru.choomandco.diplsm.storage.interfaces.
    SortedStringTable;
import ru.choomandco.diplsm.storage.memtable.MemTable;
10 import ru.choomandco.diplsm.storage.sstable.SSTable;
import ru.choomandco.diplsm.storage.sstable.SSTableMetadata;

import java.io.*;
import java.nio.file.Files;
15 import java.nio.file.Paths;
import java.util.*;
import java.util.concurrent.ConcurrentSkipListMap;
import java.util.concurrent.atomic.AtomicLong;
import java.util.stream.Collectors;

20 /**
 * Основной класс реализации LSM-хранилища.
 * Отвечает за работу с MemTable, SSTable, а также управление
 * флашами, компактацией и метаданными.
 */
25 public class StorageCore implements DipLSMStorage {
    /** Путь к директории, где хранятся SSTable-файлы */
    protected final String SSTABLE_FOLDER = "./data/lsm/tables
        /";
    /** Путь к файлу MANIFEST, содержащему информацию об уровн
        ях хранения SSTable-файлов */
    protected final String MANIFEST_PATH = "./data/lsm/
        MANIFEST";
30 /** Нулевой уровень в иерархии уровней LSM */
    protected final int LEVEL_ZERO = 0;

```

```

    /** Общее количество уровней в LSM */
    protected final int NUM_OF_LEVELS = 5;
    /** Счётчик файлов, используемый для генерации уникальных
        имён SSTable */
35    protected final AtomicLong FILE_COUNTER = new AtomicLong()
        ;
    /** Порог количества SSTable-файлов на уровень, после кото
        рого запускается компактация */
    protected int tierThreshold;

    /** MemTable      структура в памяти для временного хранени
        я данных */
40    protected MemoryTable memoryTable;
    /** Объект для работы с MANIFEST-файлом */
    protected ManifestHandler manifestHandler;
    /** Движок компактации SSTable-файлов */
    protected CompEngine compactationEngine;
45    /** Метаданные всех SSTable-файлов, отсортированные по уро
        вням */
    protected Map<Integer, TreeSet<SSTableMetadata>>
        metadataMap;
    /**
        * Объект для взаимодействия с файлами SSTable
        */
50    SortedStringTable table = new SSTable();

    /**
        * Конструктор по умолчанию.
        * Использует размер MemTable по умолчанию и порог компакт
            ации = 5.
55    */
    public StorageCore() {
        this(1024L * 1024, 5);
    }

    /**
60    * Конструктор с пользовательским размером MemTable.
        * @param memTableMaxSize максимальный размер MemTable в б
            айтах
        */
    public StorageCore(Long memTableMaxSize) {
65        this(memTableMaxSize, 5);
    }

```

```

/**
 * Основной конструктор, инициализирует хранилище, директо
    рии, читает MANIFEST и восстанавливает метаданные.
70 * @param memTableMaxSize максимальный размер MemTable
    * @param sstableTierThreshlod порог количества файлов на
        уровень до компактации
    */
public StorageCore(Long memTableMaxSize, int
    sstableTierThreshlod) {
    tierThreshold = sstableTierThreshlod;
75
    generateTableFolder();

    memoryTable = new MemTable(memTableMaxSize);

    manifestHandler = new ManifestHandler();
    manifestHandler.readManifest(SSTABLE_FOLDER,
        MANIFEST_PATH);

    metadataMap = new ConcurrentSkipListMap<>();
    for (int lvl = 0; lvl < NUM_OF_LEVELS; lvl++) {
85         this.metadataMap.put(lvl, new TreeSet<>());
    }

    for (Map.Entry<String, Integer> entry :
        manifestHandler.getFileTiers().entrySet()) {
        SSTableMetadata meta = null;
90         try {
            meta = new SSTableMetadata(entry.getKey(),
                entry.getValue(), table.readWholeIntoMap(
                    entry.getKey()).keySet());
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
95         metadataMap.get(entry.getValue()).add(meta);
    }

    compactationEngine = new CompactationEngine();

100    for (Map.Entry<Integer, TreeSet<SSTableMetadata>>
        entry : metadataMap.entrySet()) {
        while (entry.getValue().size() >= tierThreshold) {

```

```

        compactationInitialization(entry.getKey());
    }
}
105    startFlushTimer();
}

/**
 * Записывает ключ-значение в память. Если MemTable переполнена
    вызывается flush.
110 */
@Override
public void put(String key, String value) {
    if (memoryTable.put(key, value)) {
        flush(LEVEL_ZERO);
115    }
}

/**
 * Получает значение по ключу, сначала из памяти, затем из
    SSTable-файлов.
120 */
@Override
public String get(String key) {
    String memTableValue = memoryTable.get(key);
    if (memTableValue != null) {
125        return memTableValue;
    }

    try {
        for (int level : new TreeSet<>(metadataMap.keySet()
130        ())) {
            TreeSet<SSTableMetadata> levelSet =
                metadataMap.get(level);
            if (levelSet == null) continue;

            Iterator<SSTableMetadata> descendingIterator =
                levelSet.descendingIterator();
            while (descendingIterator.hasNext()) {
135                SSTableMetadata meta = descendingIterator.
                    next();
                if (meta.getBloomFilter().mightContain(key)
                    ) {

```

```

        return table.getByKey(key, meta.
            getFilename());
    }
}
}
140
} catch (Exception e) {
    System.out.println("Error while iterating - " + e
        );
    return null;
}

return null;
}

/**
 * Удаляет ключ из памяти (логическое удаление).
 */
@Override
150
public void delete(String key) {
155
    memoryTable.delete(key);
}

/**
 * Выполняет флэш MemTable на диск в SSTable-файл, после ч
    его очищает MemTable.
160
 * Также обновляет MANIFEST и метаданные.
 */
@Override
public synchronized void flush(int tier) {
165
    if (memoryTable.isEmpty()) {
        return;
    }

    Map<String, String> snapshot = new TreeMap<>(
        memoryTable.getMap());

    String tempFilename = generateNewTableName(tier) + ".
        temp";
    table.writeTableFromMap(snapshot, tempFilename);

    String finalFilename = tempFilename.replace(".temp", "
        ");

```

```

File tempFile = new File(tempFilename);
File finalFile = new File(finalFilename);
175 if (!tempFile.renameTo(finalFile)) {
    throw new RuntimeException("Failed to rename
        SSTable temp file to final file");
}

180 manifestHandler.addNewFile(finalFilename, tier,
    MANIFEST_PATH);
SSTableMetadata meta = new SSTableMetadata(
    finalFilename, tier, snapshot.keySet());
metadataMap.computeIfAbsent(tier, k -> new TreeSet<>()
    ).add(meta);

memoryTable.emptyMap();

185 checkForCompaction(LEVEL_ZERO);
}

/**
190 * Проверяет, нужно ли запускать компактацию на указанном
    уровне.
    * При необходимости вызывает рекурсивно на следующий уров
        ень.
    */
protected void checkForCompaction(int level) {
    if (metadataMap.get(level).size() >= tierThreshold) {
195 compactationInitialization(level);
        if (level + 1 != NUM_OF_LEVELS) {
            checkForCompaction(level + 1);
        }
    }
}

200 }

/**
    * Инициализирует компактацию файлов на заданном уровне.
    * Компактация перемещает данные на следующий уровень и об
        новляет метаданные.
205 */
protected void compactationInitialization(int level) {
    List<SSTableMetadata> listToCompact = metadataMap.get(
        level).stream()

        // .sorted()

```

```

210                                     .limit(tierThreshold)
                                     .collect(Collectors.toList
                                         ());

        int targetLevel = (level == NUM_OF_LEVELS - 1) ? level
            : level + 1;
        if (listToCompact.isEmpty()) {
            return;
215     }
        metadataMap.get(level).removeAll(listToCompact);
        SSTableMetadata newMeta = compactationEngine.compact(
            new ArrayList<>(listToCompact),
            generateNewTableName(targetLevel), targetLevel);
        metadataMap.get(newMeta.getTier()).add(newMeta);

220     manifestHandler.postCompactationRebuild(listToCompact,
        newMeta, MANIFEST_PATH);
    }

    //TODO удалить перед релизом
    /**
225     * Принудительная компакция нулевого уровня. Используетс
        я в отладочных целях.
        */
    public void forceCompact() {
        compactationInitialization(LEVEL_ZERO);
    }

230    /**
        * Генерирует уникальное имя нового SSTable-файла с указан
            ием уровня хранения.
        */
    protected String generateNewTableName(int tier) {
235        String timestamp = System.currentTimeMillis() + "_" +
            FILE_COUNTER.incrementAndGet();
        return SSTABLE_FOLDER + "T" + tier + "/sstable_" +
            timestamp + ".sst";
    }

    /**
240     * Запускает таймер для периодического флеша MemTable на д
        иск (каждые 3 минуты).
        */

```

```

protected void startFlushTimer() {
    Thread thread = new Thread(() -> {
        while (!Thread.currentThread().isInterrupted()) {
            try {
                Thread.sleep(180000); // 3 минуты = 180000

                if (!memoryTable.isEmpty()) {
                    flush(LEVEL_ZERO);
                }

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });

    thread.setDaemon(true);
    thread.start();
}

/**
 * Создаёт директории для всех уровней SSTable-хранилища,
 * если они ещё не существуют.
 */
protected void generateTableFolder() {
    try {
        Files.createDirectories(Paths.get(SSTABLE_FOLDER))
            ;
        for (int i = 0; i < NUM_OF_LEVELS; i++) {
            Files.createDirectories(Paths.get(
                SSTABLE_FOLDER + "T" + i));
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
}

```

Листинг П1.2

StorageCoreAsync.java - модификация ядра сервиса для многопоточной работы

```
package ru.choomandco.diplsm.storage.core;
```

```

import ru.choomandco.diplsm.storage.sstable.SSTableMetadata;
5
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
10 import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
15 import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.stream.Collectors;

/**
20 * Асинхронная реализация LSM-хранилища.
* Расширяет {@link StorageCore}, выполняя операции flush и ко
мпактацию в отдельных потоках с использованием
ExecutorService.
*/
public class StorageCoreAsync extends StorageCore{
    /** Исполнитель задач, связанных с дисковыми операциями */
25 protected final ExecutorService diskExecutor = Executors.
    newSingleThreadExecutor(r -> {
        Thread t = new Thread(r, "LSM-Disk-Worker");
        t.setDaemon(true);
        return t;
    });
30
    /**
    * Конструктор по умолчанию с размером MemTable 4 МБ и пор
    огом компактации 5.
    */
    public StorageCoreAsync() {
35         this(4L * 1024 * 1024, 5);
    }

    /**
    * Конструктор с пользовательским размером MemTable.
40 * @param memTableMaxSize максимальный размер MemTable в б
    айтах

```

```

    */
    public StorageCoreAsync(Long memTableMaxSize) {
        this(memTableMaxSize, 5);
    }

    /**
     * Основной конструктор, вызывает родительский {@link
     * StorageCore}.
     * @param memTableMaxSize максимальный размер MemTable
     * @param sstableTierThreshlod порог количества файлов на
     *        уровень до компактации
    */
    public StorageCoreAsync(Long memTableMaxSize, int
        sstableTierThreshlod) {
        super(memTableMaxSize, sstableTierThreshlod);
    }

    /**
     * Асинхронно сбрасывает MemTable в SSTable.
     * Снимок данных делается сразу, а запись и обновление мет
     * аданных происходит в фоне.
     * @param tier уровень, на который выполняется flush
    */
    @Override
    public synchronized void flush(int tier) {
        if (memoryTable.isEmpty()) return;

        Map<String,String> snapshot = new TreeMap<>()
            {
                {
                    memoryTable.getMap();
                }
            };
        String finalName = generateNewTableName(tier);
        String tempName = finalName + ".temp";
        memoryTable.emptyMap();

        diskExecutor.submit(() -> {
            //System.out.println("[disk] Starting flush tier="
            //    + tier);
            try {
                table.writeTableFromMap(snapshot, tempName);

                Path src = Paths.get(tempName);
                Path dst = Paths.get(finalName);
                boolean moved = false;
            }
        });
    }

```

```

80         for (int attempt = 1; attempt <= 5; attempt++)
            {
                try {
                    Files.move(src, dst,
                               StandardCopyOption.ATOMIC_MOVE
                               ,
                               StandardCopyOption.
                                   REPLACE_EXISTING);
                    moved = true;
                    break;
                } catch (IOException e) {
                    Thread.sleep(50);
                }
            }
            if (!moved) {
                throw new RuntimeException("Failed to move
                                           " + tempName + "      " + finalName);
            }

            manifestHandler.addNewFile(finalName, tier,
                                       MANIFEST_PATH);
            SSTableMetadata meta = new SSTableMetadata(
                finalName, tier, snapshot.keySet());
            metadataMap.get(tier).add(meta);

            checkForCompactation(LEVEL_ZERO);

            } catch (Exception ex) {
                ex.printStackTrace();
            }
        });
    }

    /**
105     * Проверяет, требуется ли компактация, и при необходимости
        * и запускает её в фоне.
        * @param level уровень, для которого проверяется необходи-
        * мость компактации
        */
    @Override
    public void checkForCompactation(int level) {
110        if (metadataMap.get(level).size() >= tierThreshold) {
            compactationInitialization(level);

```

```

    }
}

115  /**
    * Выполняет компактацию файлов заданного уровня в следующ
    * ий уровень.
    * Запускается асинхронно. При необходимости рекурсивно вы
    * зывает компактацию следующего уровня.
    * @param level уровень, с которого начинается компактация
    */
120  @Override
    public void compactationInitialization(int level) {
        List<SSTableMetadata> toCompact = metadataMap.get(
            level).stream()
                .sorted()
                .limit(tierThreshold)
125                .collect(Collectors.toList());
        if (toCompact.isEmpty()) {
            return;
        }

130        metadataMap.get(level).removeAll(toCompact);

        int nextLevel = Math.min(level + 1, NUM_OF_LEVELS - 1)
            ;
        SSTableMetadata newMeta = compactationEngine.compact(
            toCompact,
135            generateNewTableName(nextLevel),
            nextLevel
        );

        metadataMap.get(newMeta.getTier()).add(newMeta);
140        manifestHandler.postCompactationRebuild(toCompact,
            newMeta, MANIFEST_PATH);

        if (metadataMap.get(nextLevel).size() >= tierThreshold
            ) {
            checkForCompactation(nextLevel);
        }
145    }
}

```

MemTable.java

```
package ru.choomandco.diplsm.storage.memtable;

import ru.choomandco.diplsm.storage.interfaces.MemoryTable;

5
import java.nio.charset.StandardCharsets;
import java.util.*;
import java.util.concurrent.ConcurrentSkipListMap;

10 /**
 * Класс MemTable с кастомизируемым объемом памяти
 */
public class MemTable implements MemoryTable {
    /** Мапа для хранения всех ключ-значений */
    15 private ConcurrentSkipListMap<String, String> table;
    /** Максимальный размер MemTable в байтах */
    private final long maxSizeInBytes;
    /** Нынешний размер MemTable */
    private long currentSizeInBytes;

    20 /**
     * Конструктор MemTable по умолчанию, с максимальным разме
     * ром памяти в 4МБ.
     * Внутри этот метод вызывает кастомный конструктор с пред
     * записанным размером.
     */
    25 public MemTable() {
        this(4 * 1024 * 1024);
    }

    /**
    30 * Кастомный конструктор с изменяемым размером памяти
     * @param maxSizeInBytes Максимальный размер MemTable
     */
    public MemTable(long maxSizeInBytes) {
        this.table = new ConcurrentSkipListMap<>();
        35 this.maxSizeInBytes = maxSizeInBytes;
        this.currentSizeInBytes = 0;
    }

    /**
    40 * Метод, чтобы класть новую пару ключ-значение в MemTable
```

```

*
* @param key    Ключ
* @param value  Значение
* @return
45 */
@Override
public synchronized boolean put(String key, String value)
{
    int keySize = key.getBytes(StandardCharsets.UTF_8).
        length;
    int valueSize = value.getBytes(StandardCharsets.UTF_8)
        .length;
50    int entrySize = keySize + valueSize;

    String oldValue = table.put(key, value);
    if (oldValue != null) {
        currentSizeInBytes -= oldValue.getBytes(
            StandardCharsets.UTF_8).length;
55    }
    currentSizeInBytes += entrySize;

    return isFull();
}

60 /**
 * Метод для извлечения значения по ключу
 * @param key Ключ для извлечения
 * @return Значение ключа
65 */
@Override
public synchronized String get(String key) {
    return table.getDefault(key, null);
}

70 /**
 * Удаляет значение по ключу
 * @param key Ключ значения для удаления
 * /
75 */
@Override
public synchronized void delete(String key) {
    String removedValue = table.remove(key);
    if (removedValue != null) {

```

```

        currentSizeInBytes -= (key.getBytes(
            StandardCharsets.UTF_8).length +
            removedValue.getBytes(StandardCharsets.
                UTF_8).length);
    }
}

/**
 * Возвращает копию карты MemTable
 * @return Копия карты
 */
@Override
public synchronized Map<String, String> getMap() {
    return new TreeMap<String, String>(table);
}

/**
 * Метод проверяет заполненность MemTable
 * @return Значение выше лимита или нет (true или false)
 */
@Override
public boolean isFull() {
    return currentSizeInBytes >= maxSizeInBytes;
}

/**
 * Метод очистки карты
 */
@Override
public void emptyMap() {
    currentSizeInBytes = 0;
    table.clear();
}

@Override
public boolean isEmpty() {
    return table.isEmpty();
}
}

```

Листинг П1.4

SSTable.java

```
package ru.choomandco.diplsm.storage.sstable;
```

```

import ru.choomandco.diplsm.storage.interfaces.
    SortedStringTable;

5
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
10 import java.nio.file.Paths;
import java.util.*;

import static java.nio.charset.StandardCharsets.UTF_8;

15 /**
 * Класс SSTable, работающий с бинарным текстом
 */
public class SSTable implements SortedStringTable {
    private static final int FOOTER_SIZE = 12;
20 private static final int MAGIC = 0x4C534D31; // 'LSM1'
    /**
     * Метод пишет карту MemTable в новый SSTable.
     * @param memTableMap Мапа с данными из MemTable
     * @param filename Название файла SSTable
25 */
    @Override
    public void writeTableFromMap(Map<String, String>
        memTableMap, String filename) {
        try (RandomAccessFile raf = new RandomAccessFile(
            filename, "rw")) {
30 raf.setLength(0);
            raf.seek(12);

            List<IndexEntry> index = new ArrayList<>();
            for (Map.Entry<String, String> e : memTableMap.
                entrySet()) {
                long pos = raf.getFilePointer();
35 byte[] key = e.getKey().getBytes(UTF_8);
                byte[] val = e.getValue().getBytes(UTF_8);

                raf.writeInt(key.length);
                raf.write(key);
40 raf.writeInt(val.length);
                raf.write(val);
            }
        }
    }
}

```

```

        index.add(new IndexEntry(e.getKey(), pos));
    }

    long indexOffset = raf.getFilePointer();
    raf.writeInt(index.size());
    for (IndexEntry ie : index) {
        byte[] key = ie.getKey().getBytes(UTF_8);
        raf.writeInt(key.length);
        raf.write(key);
        raf.writeLong(ie.getOffset());
    }

    raf.seek(0);
    raf.writeLong(indexOffset);
    raf.writeInt(0x4C534D31);
} catch (IOException e) {
    throw new RuntimeException(e);
}

/**
 * Метод для чтения из SSTable по ключу
 * @param key Ключ
 * @return Значение по ключу
 */
@Override
public String getByKey(String key, String filename) {
    try (RandomAccessFile raf = new RandomAccessFile(
        filename, "r")) {
        raf.seek(0);
        long indexOffset = raf.readLong();
        int magic = raf.readInt();
        if (magic != 0x4C534D31) throw new IOException("
            Bad SSTable magic");

        raf.seek(indexOffset);
        int count = raf.readInt();

        List<IndexEntry> idx = new ArrayList<>(count);
        for (int i = 0; i < count; i++) {
            int klen = raf.readInt();

```

```

        byte[] kbs = new byte[klen]; raf.readFully(kbs
        );
        long off = raf.readLong();
        idx.add(new IndexEntry(new String(kbs, UTF_8),
            off));
85    }
        int pos = Collections.binarySearch(
            idx,
            new IndexEntry(key, -1),
            Comparator.comparing(IndexEntry::getKey)
90    );
        if (pos < 0) return null;
        long dataOff = idx.get(pos).getOffset();

        raf.seek(dataOff);
95    int klen = raf.readInt();
        raf.skipBytes(klen);
        int vlen = raf.readInt();
        byte[] vbs = new byte[vlen]; raf.readFully(vbs);
        return new String(vbs, UTF_8);
100    } catch (FileNotFoundException e) {
        throw new RuntimeException(e);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
105    }

/**
 * Читает весь SSTable-файл в отсортированное отображение
 *   ключ значение.
 * Формат бинарного файла:
110 * [Data Block][Index Block][Footer]
 * Footer (12 байт) = [indexOffset (8 байт)][magic (4 байт
 *   a)]
 *
 * @param filename путь к SSTable-файлу
 * @return TreeMap с данными из файла
115 * @throws IOException при ошибках I/O или некорректном фо
 *   рмате
 */
@Override
public Map<String, String> readWholeIntoMap(String
    filename) throws IOException {

```

```

120     Map<String,String> result = new TreeMap<>();
    try (RandomAccessFile raf = new RandomAccessFile(
        filename, "r")) {

        raf.seek(0);
        long indexOffset = raf.readLong();
        int magic = raf.readInt();
125     if (magic != 0x4C534D31) throw new IOException("
        Invalid SSTable file (magic mismatch): " +
            filename);

        raf.seek(FOOTER_SIZE);
        while (raf.getFilePointer() < indexOffset) {
            int keyLen = raf.readInt();
            byte[] keyBytes = new byte[keyLen];
130         raf.readFully(keyBytes);

            int valLen = raf.readInt();
            byte[] valBytes = new byte[valLen];
135         raf.readFully(valBytes);

            String key = new String(keyBytes,
                StandardCharsets.UTF_8);
            String val = new String(valBytes,
                StandardCharsets.UTF_8);
            result.put(key, val);
140         }
        }
        return result;
    }

145 /**
    * Читает все строки SSTable-файла и возвращает список стр
    * ок вида "key=value".
    * Формирует список в том порядке, в котором записаны пары
    * в Data Block.
    *
    * @param filename путь к SSTable-файлу
150 * @return List<String> всех записей "key=value"
    * @throws IOException при ошибках I/O или некорректном фо
    * рмате
    */
    @Override

```

```

public List<String> readStringsIntoList(String filename)
    throws IOException {
155     List<String> result = new ArrayList<>();
        try (RandomAccessFile raf = new RandomAccessFile(
            filename, "r")) {
            long fileSize = raf.length();
            raf.seek(fileSize - FOOTER_SIZE);
            long indexOffset = raf.readLong();
160         int magic = raf.readInt();
            if (magic != MAGIC) {
                throw new IOException("Invalid SSTable file (
                    magic mismatch): " + filename);
            }

165         raf.seek(FOOTER_SIZE);
            while (raf.getFilePointer() < indexOffset) {
                int keyLen = raf.readInt();
                byte[] keyBytes = new byte[keyLen];
                raf.readFully(keyBytes);

170                 int valLen = raf.readInt();
                byte[] valBytes = new byte[valLen];
                raf.readFully(valBytes);

175                 String key = new String(keyBytes,
                    StandardCharsets.UTF_8);
                String val = new String(valBytes,
                    StandardCharsets.UTF_8);
                result.add(key + "=" + val);
            }
        }
180     return result;
}

/**
 * Удаляет файл SSTable.
185  *
 * @param filename путь к файлу
 * @throws IOException при ошибке удаления
 */
@Override
190 public void deleteFile(String filename) throws IOException
    {

```

```

        Path p = Paths.get(filename).toAbsolutePath().
            normalize();
        try {
            Files.delete(p);
//            System.out.println("[delete] Deleted SSTable " +
p);
195    } catch (IOException e) {
//            System.err.println("[delete] FAILED to delete
SSTable " + p + ": " + e.getMessage());
            throw e;
        }
    }
}

```

Листинг П1.5

SSTableMetadata.java

```

package ru.choomandco.diplsm.storage.sstable;

import ru.choomandco.diplsm.storage.bloomfilter.BloomFilter;
5
import java.util.Collections;
import java.util.Objects;
import java.util.Set;
import java.util.function.Function;
10
public class SSTableMetadata implements Comparable<
SSTableMetadata> {
    private final String filename;
    private int tier;
    private String minKey;
15    private String maxKey;
    private BloomFilter<String> bloomFilter;

    public SSTableMetadata(String filename, int tier, Set<
String> keySet) {
        this.filename = filename;
20        this.tier = tier;
        this.minKey = Collections.min(keySet);
        this.maxKey = Collections.max(keySet);

        Function<String, Integer> hash1 = String::hashCode;
25        Function<String, Integer> hash2 = s -> s.hashCode() *
            31;
    }
}

```

```

        this.bloomFilter = new BloomFilter<String>(1024, new
            Function[]{ hash1, hash2 });

        bloomFilter.addKeysFromMap(keySet);
30    }

    @Override
    public int compareTo(SSTableMetadata other) {
        if (Objects.equals(this.filename, other.getFilename()))
            {
35                return 0;
            }

        return Long.compare(extractTimestamp(this.filename),
            extractTimestamp(other.filename));
    }
40

    private long extractTimestamp(String filename) {
        String[] parts = filename.split("_");
        return Long.parseLong(parts[1]);
    }
45

    public String getFilename() {
        return filename;
    }

    public int getTier() {
50        return tier;
    }

    public String getMinKey() {
55        return minKey;
    }

    public String getMaxKey() {
        return maxKey;
    }
60

    public BloomFilter<String> getBloomFilter() {
        return bloomFilter;
    }

65    public void increaseTier() {

```

```

        tier++;
    }

70    public void decreaseTier() {
        tier--;
    }

    public void setMinKey(String minKey) {
75        this.minKey = minKey;
    }

    public void setMaxKey(String maxKey) {
        this.maxKey = maxKey;
80    }

    public void setBloomFilter(BloomFilter<String> bloomFilter
        ) {
        this.bloomFilter = bloomFilter;
    }
85 }

```

Листинг П1.6

CompactationEngine.java

```

package ru.choomandco.diplsm.storage.compengine;

import ru.choomandco.diplsm.storage.interfaces.CompEngine;
5 import ru.choomandco.diplsm.storage.interfaces.
    SortedStringTable;
import ru.choomandco.diplsm.storage.sstable.SSTable;
import ru.choomandco.diplsm.storage.sstable.SSTableMetadata;

import java.io.IOException;
10 import java.util.*;

/**
 * Класс, реализующий движок компактации для LSM-структуры хра
    нения.
 * Объединяет несколько SSTable-файлов в один, устраняя дублик
    аты ключей
15 * и освобождая место за счёт удаления устаревших файлов.
 */
public class CompactationEngine implements CompEngine {
    protected SortedStringTable table ;

```

```

20 public CompactationEngine() {
    table = new SSTable();
}

/**
25 * Выполняет компактацию заданного списка SSTable-файлов.
* Все ключи и значения из указанных таблиц объединяются,
* при этом в случае дублирования ключей остаётся последне
  е значение (по порядку в списке).
* Создаётся новый SSTable, старые файлы удаляются.
*
30 * @param tablesMeta список метаданных SSTable-файлов, под
  лежащих компактации
* @param fileToCompact имя нового SSTable-файла, в которы
  й будут записаны данные
* @param level уровень, на который будет записан результа
  т компактации
* @return метайнформация о новом SSTable-файле
* @throws IllegalArgumentException если список таблиц пус
  т
35 * @throws RuntimeException если не удалось удалить один и
  з исходных файлов
*/
public SSTableMetadata compact(List<SSTableMetadata>
  tablesMeta, String fileToCompact, int level) {
    if (tablesMeta.isEmpty()) {
        throw new IllegalArgumentException("No SSTables
          provided for compaction");
40    }

    Map<String, String> allEntries = new TreeMap<>();
    for (SSTableMetadata meta : tablesMeta) {
        try {
45            allEntries.putAll(table.readWholeIntoMap(meta.
              getFilename()));
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
50
    table.writeTableFromMap(allEntries, fileToCompact);

```

```

        for (SSTableMetadata file : tablesMeta) {
            try {
55 //                System.out.println("[compact] About to
delete: " + file.getFilename() + " (exists=" + Files.exists
(Paths.get(file.getFilename())) + ")");
                table.deleteFile(file.getFilename());
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
60        }

        return new SSTableMetadata(fileToCompact, level,
            allEntries.keySet());
    }
}

```

Листинг П1.7

ManifestHandler.java

```

package ru.choomandco.diplsm.storage.core;

import ru.choomandco.diplsm.storage.sstable.SSTableMetadata;
5
import java.io.*;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
10 import java.util.Map;
import java.util.zip.CRC32;

class ManifestHandler {
    private final Map<String, Integer> fileTiers = new HashMap
        <>();
15

    public Map<String, Integer> getFileTiers() {
        return fileTiers;
    }

    /**
20     * Метод читает файл MANIFEST. В случае, если файл не суще
        ствует, метод создает файл и
        * сканирует существующие папки и файлы и записывает новый
        файл MANIFEST, если необходимо
        */
}

```

```

public void readManifest(String tablesPath, String
manifestPath) {
25     File file = new File(manifestPath);

    if (file.exists()) {
        if (!loadAndVerifyManifest(manifestPath)) {
            System.out.println("MANIFEST corrupted or
invalid CRC, rebuilding...");
30             rebuildManifest(tablesPath, manifestPath);
        }
    } else {
        System.out.println("No MANIFEST found, rebuilding
...");
        rebuildManifest(tablesPath, manifestPath);
35     }
}

/**
 * Метод загружает информацию из MANIFEST
40 */
private boolean loadAndVerifyManifest(String manifestPath)
{
    try (BufferedReader reader = new BufferedReader(new
FileReader(manifestPath))) {
        String crcLine = reader.readLine();
        if (crcLine == null || !crcLine.startsWith("#CRC="
)) return false;
45

        long expectedCRC = Long.parseLong(crcLine.
substring(5));
        CRC32 crc = new CRC32();

        String line;
50        List<String> entries = new ArrayList<>();
        while ((line = reader.readLine()) != null) {
            crc.update(line.getBytes());
            entries.add(line);
        }

55        if (crc.getValue() != expectedCRC) return false;

        for (String entry : entries) {
            String[] parts = entry.split(" ");

```



```

        fileTiers.put(dir + "\\\" + sstable
            .getName(), tier);
    }
}
}
}

writeManifest(manifestPath);
}

/**
 * Записывает текущее состояние уровней файла в
 * SSTABLE_MANIFEST_FILE.
 *
 * 105 * Метод перебирает карту fileLevels и записывает имя кажд
 *      ого файла SSTable
 * * вместе с его соответствующим уровнем в файле манифеста.
 * * Файл манифеста полностью перезаписывается обновленным с
 *      одержимым.
 *
 * * Этот метод обычно вызывается, когда необходимо записать
 *      структуру файла или
 *
 * 110 * когда необходимо сгенерировать манифест с нуля.
 *
 * * @throws RuntimeException если при записи файла манифест
 *      а произошла ошибка ввода-вывода
 * */
private void writeManifest(String manifestPath) {
    115     List<String> entries = new ArrayList<>();
    for (Map.Entry<String, Integer> entry : fileTiers.
        entrySet()) {
        entries.add("T" + entry.getValue() + " " + entry.
            getKey());
    }

    120     CRC32 crc = new CRC32();
    for (String entry : entries) {
        crc.update(entry.getBytes());
    }

    125     try (BufferedWriter writer = new BufferedWriter(new
        FileWriter(manifestPath))) {
        writer.write("#CRC=" + crc.getValue());
    }
}

```

```

        writer.newLine();
        for (String entry : entries) {
            writer.write(entry);
            writer.newLine();
        }
    } catch (IOException e) {
        throw new RuntimeException("Error writing MANIFEST
            ", e);
    }
}

/**
 * Обновление уровня файла внутри манифеста
 * @param filename название файла
 * @param newTier назначаемый уровень
 * @param manifestPath путь до файла манифеста
 */
public void updateFileTier(String filename, int newTier,
    String manifestPath) {
    fileTiers.put(filename, newTier);
    writeManifest(manifestPath);
}

public void addNewFile(String filename, int tier, String
    manifestPath) {
    fileTiers.put(filename, tier);
    writeManifest(manifestPath);
}

public void postCompactationRebuild(List<SSTableMetadata>
    listToDelete, SSTableMetadata newFile, String
    manifestPath) {
    fileTiers.put(newFile.getFilename(), newFile.getTier()
        );
    for (SSTableMetadata meta : listToDelete) {
        fileTiers.remove(meta.getFilename());
    }
    writeManifest(manifestPath);
}
}

```

Листинг П1.8

```

package ru.choomandco.diplsm.storage.bloomfilter;

import java.util.BitSet;
5 import java.util.Set;
import java.util.function.Function;

/**
 * Простая реализация фильтра Блума      вероятностной структур
 * ы данных,
10 * предназначенной для проверки принадлежности элемента множес
    тву.
 *
 * @param <T> тип элементов, добавляемых в фильтр
 */
public class BloomFilter<T> {
15     /** Внутреннее битовое представление фильтра */
    private final BitSet bitSet;
    /** Массив хеш-функций, используемых для установки битов
        */
    private final Function<T, Integer>[] hashFunctions;

20     /**
        * Создаёт новый фильтр Блума заданного размера и с переда
            нным набором хеш-функций.
        *
        * @param size размер битового массива
        * @param hashFunctions массив хеш-функций, применяемых к
            ключам
25     */
    public BloomFilter(int size, Function<T, Integer>[]
        hashFunctions) {
        this.bitSet = new BitSet(size);
        this.hashFunctions = hashFunctions;
    }

30     /**
        * Добавляет ключ в фильтр. Для каждого ключа используются
            все переданные хеш-функции.
        *
        * @param key ключ, который нужно добавить
35     */
    public void add(T key) {

```

```

        for (Function<T, Integer> hashFunction : hashFunctions
            ) {
            int hash = hashFunction.apply(key);
            bitSet.set(Math.abs(hash) % bitSet.size(), true);
40    }
    }

    /**
     * Проверяет, может ли фильтр содержать данный ключ.
     * Возвращает {@code true}, если ключ, возможно, присутств
        ует;
     * {@code false}, если точно отсутствует.
     *
     * @param key ключ для проверки
     * @return {@code true} если ключ может присутствовать, {
        @code false}      если точно отсутствует
50    */
    public boolean mightContain(T key) {
        for (Function<T, Integer> hashFunction : hashFunctions
            ) {
            int hash = hashFunction.apply(key);
            if (!bitSet.get(Math.abs(hash) % bitSet.size())) {
55                return false;
            }
        }
        return true;
    }

    /**
     * Добавляет все ключи из заданного множества в фильтр.
     * Использует все хеш-функции для каждого ключа.
     *
     * @param keySet множество ключей, которые необходимо доба
        вить
65    */
    public void addKeysFromMap(Set<T> keySet) {
        for (T key : keySet) {
            for (Function<T, Integer> hashFunction :
                hashFunctions) {
70                int hash = hashFunction.apply(key);
                bitSet.set(Math.abs(hash) % bitSet.size(),
                    true);
            }
        }
    }
}

```

```

    }
}
75 }
```

Листинг П1.9

DipLSMStorage.java - интерфейс ядра

```

package ru.choomandco.diplsm.storage.interfaces;

public interface DipLSMStorage {
5     void put(String key, String value);
        String get(String key);
        void delete(String key);
        void flush(int level);
}

```

Листинг П1.10

SortedStringTable.java - интерфейс SSTable

```

package ru.choomandco.diplsm.storage.interfaces;

import java.io.IOException;
5 import java.util.List;
import java.util.Map;

public interface SortedStringTable {
    void writeTableFromMap(Map<String, String> memTable,
        String filename);
10    String getByKey(String key, String filename);
    Map<String, String> readWholeIntoMap(String filename)
        throws IOException;
    List<String> readStringsIntoList(String filename) throws
        IOException;
    void deleteFile(String filename) throws IOException;
}

```

Листинг П1.11

MemoryTable.java - интерфейс MemTable

```

package ru.choomandco.diplsm.storage.interfaces;

import java.util.Map;
5

public interface MemoryTable {
    boolean put(String key, String value);
}

```

```

    void delete(String key);
    String get(String key);
10 Map<String, String> getMap();
    boolean isFull();
    void emptyMap();
    boolean isEmpty();
}

```

Листинг П1.12

InvalidCRC.java - исключение по CRC

```

package ru.choomandco.diplsm.exception.invalid.crc;

public class InvalidCRC extends Exception{
5     public InvalidCRC(String message) {
        super(message);
    }
}

```

Листинг П1.13

PlainTextSSTable.java - для работы с не бинарными SSTable

```

package ru.choomandco.diplsm.storage.sstable;

import ru.choomandco.diplsm.exception.invalid.crc.InvalidCRC;
5 import ru.choomandco.diplsm.storage.interfaces.
    SortedStringTable;

import java.io.*;
import java.nio.file.Files;
import java.nio.file.Paths;
10 import java.util.*;
import java.util.zip.CRC32;

/**
 * Класс SSTable, работающий с чистым текстом
15 */
public class PlainTextSSTable implements SortedStringTable {
    /**
     * Метод пишет mapу MemTable в новый SSTable.
     * @param memTableMap Мапа с данными из MemTable
     * @param filename Название файла SSTable
20 */
    @Override

```

```

public void writeTableFromMap(Map<String, String>
    memTableMap, String filename) {
    try (BufferedWriter writer = new BufferedWriter(new
        FileWriter(filename))) {
25         CRC32 crc = new CRC32();
        StringBuilder contentBuffer = new StringBuilder();

        for (Map.Entry<String, String> entry : memTableMap
            .entrySet()) {
            String line = entry.getKey() + ":" + entry.
                getValue();
30             contentBuffer.append(line).append("\n");
        }

        byte[] dataBytes = contentBuffer.toString().
            getBytes();
        crc.update(dataBytes);

35         writer.write("#CRC=" + crc.getValue());
        writer.newLine();

        writer.write(contentBuffer.toString());

40     } catch (IOException e) {
        throw new RuntimeException("Failed to
            writeTableFromMap SSTable", e);
    }
}

45 /**
 * Метод для чтения из SSTable по ключу
 * @param key Ключ
 * @return Значение по ключу
50 */
@Override
public String getByKey(String key, String filename) {
    try (BufferedReader reader = new BufferedReader(new
        FileReader(filename))) {
        verifyCRC(reader, filename);
55         String line;
        while ((line = reader.readLine()) != null) {
            String[] parts = line.split(":", 2);

```

```

        if (parts.length == 2 && parts[0].equals(key))
        {
            return parts[1];
        }
    }
} catch (InvalidCRC e) {
    throw new RuntimeException(e);
} catch (IOException e) {
    throw new RuntimeException("Error reading SSTable:
65         " + filename, e);
}
return null;
}

70 /**
 * Метод для чтения всего SSTable в карту
 * @param filename Название файла SSTable
 * @return Карта данных из SSTable
 */
75 @Override
public Map<String, String> readWholeIntoMap(String
    filename) {
    Map<String, String> result = new TreeMap<>();

    try (BufferedReader reader = new BufferedReader(new
        FileReader(filename))) {
80         verifyCRC(reader, filename);
        String line;
        while ((line = reader.readLine()) != null) {
            String[] parts = line.split(":", 2);
            if (parts.length == 2) {
85                 result.put(parts[0], parts[1]);
            }
        }
    } catch (InvalidCRC e) {
        throw new RuntimeException(e);
    } catch (IOException e) {
90         throw new RuntimeException("Error while reading
            SSTable into map: " + filename, e);
    }

    return result;
95 }

```

```

100  /**
    * Метод для чтения всех строк SSTable в список строк
    * @param filename Название файла SSTable
    * @return Список строк файла SSTable
    */
    @Override
    public List<String> readStringsIntoList(String filename) {
        List<String> result = new ArrayList<>();

        try (BufferedReader reader = new BufferedReader(new
            FileReader(filename))) {
            verifyCRC(reader, filename);
            String line;
            while ((line = reader.readLine()) != null) {
110         result.add(line.replace(":", "="));
            }
        } catch (InvalidCRC e) {
            throw new RuntimeException(e);
        } catch (IOException e) {
115         throw new RuntimeException("Error reading SSTable
            lines from file: " + filename, e);
        }

        return result;
    }

120
    /**
    * Метод для удаления файла SSTable
    * @param filename имя файла для удаления
    * @throws IOException ошибка удаления
125  */
    @Override
    public void deleteFile(String filename) throws IOException
    {
        Files.delete(Paths.get(filename));
    }

130
    /**
    * Валидация контрольной суммы файла SSTable
    * @param reader буферизированный "читатель" файлов
    * @param filename название файла SSTable для чтения

```

```

135     * @throws InvalidCRC ошибка некорректной контрольной сумм
        ы
    */
    private void verifyCRC(BufferedReader reader, String
        filename) throws InvalidCRC {
        try {
            reader.mark(1024 * 1024);

140            String crcLine = reader.readLine();
            if (crcLine == null || !crcLine.startsWith("#CRC="
                )) {
                throw new InvalidCRC("Missing or malformed CRC
                    line in file: " + filename);
            }

145            long expectedCrc = Long.parseLong(crcLine.
                substring(5));
            StringBuilder content = new StringBuilder();
            String line;

150            while ((line = reader.readLine()) != null) {
                content.append(line).append("\n");
            }

            CRC32 crc = new CRC32();
155            crc.update(content.toString().getBytes());
            long actualCrc = crc.getValue();

            if (actualCrc != expectedCrc) {
                throw new InvalidCRC("CRC mismatch in file: "
                    + filename +
160                    ", expected=" + expectedCrc + ",
                        actual=" + actualCrc);
            }

            reader.reset();

165            reader.readLine();

        } catch (IOException e) {
            throw new RuntimeException("Error during CRC
                verification in file: " + filename, e);
        }
    }

```

```

170     }
    }

```

Листинг П1.14

PlainTextCompactionEngine.java - для компактации бинарных SSTable

```

package ru.choomandco.diplsm.storage.compengine;

import ru.choomandco.diplsm.storage.interfaces.CompEngine;
5 import ru.choomandco.diplsm.storage.sstable.PlainTextSSTable;

public class PlainTextCompactionEngine extends
    CompactationEngine implements CompEngine {
    public PlainTextCompactionEngine() {
        table = new PlainTextSSTable();
10    }
}

```

Листинг П1.15

SSTableTest.java - для тестирования работы с файлами SSTable

```

package ru.choomandco.diplsm.storage.sstable;

import org.junit.jupiter.api.AfterEach;
5 import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
10 import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.HashMap;
15 import java.util.List;
import java.util.Map;

import static org.junit.jupiter.api.Assertions.*;

20 class SSTableTest {
    private static final String TEST_FILE = "./data/lsm/tables
        /T0/sstable_test_0.dat";
    private SSTable sstable;

    @BeforeEach

```

```

25     void setUp() {
        sstable = new SSTable();
    }

    @AfterEach
30     void tearDown() throws IOException {
        Files.deleteIfExists(Paths.get(TEST_FILE));    }

    @Test
    void testWriteAndReadWholeMap() {
35        Map<String, String> data = new HashMap<>();
        data.put("key1", "value1");
        data.put("key2", "value2");

        sstable.writeTableFromMap(data, TEST_FILE);
40        Map<String, String> result = null;
        try {
            result = sstable.readWholeIntoMap(TEST_FILE);
        } catch (IOException e) {
            throw new RuntimeException(e);
45        }

        assertEquals(2, result.size());
        assertEquals("value1", result.get("key1"));
        assertEquals("value2", result.get("key2"));
50    }

    @Test
    void testGetByKeyExisting() {
        Map<String, String> data = new HashMap<>();
55        data.put("alpha", "beta");
        sstable.writeTableFromMap(data, TEST_FILE);

        String value = sstable.getKeyBy("alpha", TEST_FILE);
        assertEquals("beta", value);
60    }

    @Test
    void testGetByKeyNonExisting() {
        Map<String, String> data = new HashMap<>();
65        data.put("foo", "bar");
        sstable.writeTableFromMap(data, TEST_FILE);

```

```

    String value = sstable.getByKey("unknown", TEST_FILE);
    assertNull(value);
70 }

@Test
void testReadStringsIntoList() {
    Map<String, String> data = new HashMap<>();
    data.put("one", "1");
75 data.put("two", "2");
    sstable.writeTableFromMap(data, TEST_FILE);

    List<String> lines = null;
    try {
        lines = sstable.readStringsIntoList(TEST_FILE);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
80
    assertTrue(lines.stream().anyMatch(l -> l.equals("one
        =1"))));
    assertTrue(lines.stream().anyMatch(l -> l.equals("two
        =2"))));
85
}

@Test
void testDeleteFile() throws IOException {
    Map<String, String> data = new HashMap<>();
    data.put("k", "v");
    sstable.writeTableFromMap(data, TEST_FILE);
    assertTrue(Files.exists(Paths.get(TEST_FILE)));
90
    sstable.deleteFile(TEST_FILE);
    assertFalse(Files.exists(Paths.get(TEST_FILE)));
95
}

@Test
void testInvalidCRCThrows() throws IOException {
    Map<String, String> data = new HashMap<>();
    data.put("a", "b");
    sstable.writeTableFromMap(data, TEST_FILE);
    Files.write(Paths.get(TEST_FILE), "corrupt".getBytes(
100
        StandardCharsets.UTF_8), StandardOpenOption.APPEND)
    ;
105

```

```

        assertThrows(RuntimeException.class, () -> sstable.
            getByKey("a", TEST_FILE));
        assertThrows(RuntimeException.class, () -> sstable.
            readWholeIntoMap(TEST_FILE));
        assertThrows(RuntimeException.class, () -> sstable.
            readStringsIntoList(TEST_FILE));
110    }
    }
}

```

Листинг П1.16

ManifestHandlerTest.java - для тестирования работы с манифестом

```

package ru.choomandco.diplsm.storage.core;

import org.apache.commons.io.FileUtils;
import org.junit.jupiter.api.AfterEach;
5 import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import ru.choomandco.diplsm.storage.interfaces.
    SortedStringTable;
import ru.choomandco.diplsm.storage.sstable.SSTable;
10 import ru.choomandco.diplsm.storage.sstable.SSTableMetadata;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
15 import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.*;

import static org.junit.jupiter.api.Assertions.*;
20

class ManifestHandlerTest {
    private static final String TEST_DIR = ".\\data_test\\lsm
        \\tables\\";
    private static final String MANIFEST_PATH = "./data_test/
        lsm/MANIFEST";
    private ManifestHandler handler;
25

    @BeforeEach
    void construct() throws IOException {
        Path baseDir = Paths.get(TEST_DIR);
        if (Files.exists(baseDir)) {
30            FileUtils.deleteDirectory(baseDir.toFile());
        }
    }
}

```

```

    }
    Files.createDirectories(baseDir.resolve("T1"));
    Files.createDirectories(baseDir.resolve("T2"));

35     handler = new ManifestHandler();
}

@AfterEach
void deconstruct() throws IOException {
40     FileUtils.deleteDirectory(new File(".\\data_test"));
    Files.deleteIfExists(Paths.get(MANIFEST_PATH));
}

@Test
45 void testRebuildManifestCreatesEntries() throws
    IOException {
    String file1 = TEST_DIR + "T1\\a.dat";
    String file2 = TEST_DIR + "T2\\b.dat";

    Files.write(Paths.get(file1), Collections.singleton(""));
50     Files.write(Paths.get(file2), Collections.singleton(""));

    handler.readManifest(TEST_DIR, MANIFEST_PATH);
    Map<String, Integer> tiers = handler.getFileTiers();
    assertTrue(tiers.get(file1).equals(1));
55     assertTrue(tiers.get(file2).equals(2));
}

@Test
void testLoadAndVerifyManifest() throws IOException {
60     String dummyName = TEST_DIR + "T1\\test_file.dat";
    Map<String, String> dummyMap = new HashMap<>();
    dummyMap.put("ke1", "val1");

    SortedStringTable dummyTable = new SSTable();
65     dummyTable.writeTableFromMap(dummyMap, dummyName);

    handler.readManifest(TEST_DIR, MANIFEST_PATH);
    dummyTable.deleteFile(dummyName);
    assertTrue(handler.getFileTiers().containsKey(
        dummyName));
}

```

```

70     }

    @Test
    void testUpdateAndAddNewFile() {
        handler.addNewFile("c.dat", 1, MANIFEST_PATH);
75         assertEquals(1, handler.getFileTiers().get("c.dat"));

        handler.updateFileTier("c.dat", 2, MANIFEST_PATH);
        assertEquals(2, handler.getFileTiers().get("c.dat"));
    }

80     @Test
    void testPostCompactionRebuild() {
        Map<String, String> dummyMap = new HashMap<>();
        dummyMap.put("key1", "val1");
85         dummyMap.put("key2", "val2");
        Set<String> dummySet = dummyMap.keySet();

        SSTableMetadata old1 = new SSTableMetadata("./data/lsm
            /tables/Ttest/old1.dat", 1, dummySet);
        SSTableMetadata old2 = new SSTableMetadata("./data/lsm
            /tables/Ttest/old2.dat", 1, dummySet);
90         SSTableMetadata newFile = new SSTableMetadata("./data/
            lsm/tables/Ttest/new.dat", 2, dummySet);
        handler.addNewFile(old1.getFilename(), old1.getTier(),
            MANIFEST_PATH);
        handler.addNewFile(old2.getFilename(), old2.getTier(),
            MANIFEST_PATH);

        List<SSTableMetadata> l = new LinkedList<>();
95         l.add(old1);
        l.add(old2);
        handler.postCompactionRebuild(l, newFile,
            MANIFEST_PATH);
        Map<String, Integer> tiers = handler.getFileTiers();
        assertFalse(tiers.containsKey(old1.getFilename()));
        assertFalse(tiers.containsKey(old2.getFilename()));
100        assertEquals(2, tiers.get(newFile.getFilename()));
    }
}

```

Листинг П1.17

```

package ru.choomandco.diplsm.storage.sstable;

import org.junit.jupiter.api.AfterEach;
5 import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
10 import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.HashMap;
15 import java.util.List;
import java.util.Map;

import static org.junit.jupiter.api.Assertions.*;

20 class SSTableTest {
    private static final String TEST_FILE = "./data/lsm/tables
        /T0/sstable_test_0.dat";
    private SSTable sstable;

    @BeforeEach
25 void setUp() {
        sstable = new SSTable();
    }

    @AfterEach
30 void tearDown() throws IOException {
        Files.deleteIfExists(Paths.get(TEST_FILE));    }

    @Test
    void testWriteAndReadWholeMap() {
35        Map<String, String> data = new HashMap<>();
        data.put("key1", "value1");
        data.put("key2", "value2");

        sstable.writeTableFromMap(data, TEST_FILE);
40        Map<String, String> result = null;
        try {
            result = sstable.readWholeIntoMap(TEST_FILE);
        } catch (IOException e) {

```

```

        throw new RuntimeException(e);
45     }

        assertEquals(2, result.size());
        assertEquals("value1", result.get("key1"));
        assertEquals("value2", result.get("key2"));
50     }

    @Test
    void testGetByKeyExisting() {
        Map<String, String> data = new HashMap<>();
55         data.put("alpha", "beta");
        sstable.writeTableFromMap(data, TEST_FILE);

        String value = sstable.getKey("alpha", TEST_FILE);
        assertEquals("beta", value);
60     }

    @Test
    void testGetByKeyNonExisting() {
        Map<String, String> data = new HashMap<>();
65         data.put("foo", "bar");
        sstable.writeTableFromMap(data, TEST_FILE);

        String value = sstable.getKey("unknown", TEST_FILE);
        assertNull(value);
70     }

    @Test
    void testReadStringsIntoList() {
        Map<String, String> data = new HashMap<>();
75         data.put("one", "1");
        data.put("two", "2");
        sstable.writeTableFromMap(data, TEST_FILE);

        List<String> lines = null;
80         try {
            lines = sstable.readStringsIntoList(TEST_FILE);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }

85         assertTrue(lines.stream().anyMatch(l -> l.equals("one
            =1"))));

```

```

        assertTrue(lines.stream().anyMatch(l -> l.equals("two
            =2"))));
    }

    @Test
    void testDeleteFile() throws IOException {
        Map<String, String> data = new HashMap<>();
        data.put("k", "v");
        sstable.writeTableFromMap(data, TEST_FILE);
        assertTrue(Files.exists(Paths.get(TEST_FILE)));

        sstable.deleteFile(TEST_FILE);
        assertFalse(Files.exists(Paths.get(TEST_FILE)));
    }

    @Test
    void testInvalidCRCThrows() throws IOException {
        Map<String, String> data = new HashMap<>();
        data.put("a", "b");
        sstable.writeTableFromMap(data, TEST_FILE);
        Files.write(Paths.get(TEST_FILE), "corrupt".getBytes(
            StandardCharsets.UTF_8), StandardOpenOption.APPEND)
            ;

        assertThrows(RuntimeException.class, () -> sstable.
            getByKey("a", TEST_FILE));
        assertThrows(RuntimeException.class, () -> sstable.
            readWholeIntoMap(TEST_FILE));
        assertThrows(RuntimeException.class, () -> sstable.
            readStringsIntoList(TEST_FILE));
    }
}

```

Листинг П1.18

MixedLoadTest.java - смешанное нагрузочное тестирование, операции put и get

```

package ru.choomandco.diplsm;

import ru.choomandco.diplsm.storage.core.StorageCoreAsync;
import ru.choomandco.diplsm.storage.interfaces.DipLSMStorage;

import java.io.File;
import java.util.Objects;
import java.util.Random;

```

```

10 import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;
import java.util.concurrent.atomic.AtomicLong;

15 public class MixedLoadTest {
    private final DipLSMStorage storage;
    private final long durationMs;
    private final int checkEvery;

    private final AtomicLong puts = new AtomicLong();
    private final AtomicLong gets = new AtomicLong();
    private final AtomicLong mismatches = new AtomicLong();
    private final AtomicLong totalPutNs = new AtomicLong();
    private final AtomicLong totalGetNs = new AtomicLong();

25     public MixedLoadTest(DipLSMStorage storage, long
        durationMs, int checkEvery) {
        this.storage = storage;
        this.durationMs = durationMs;
        this.checkEvery = checkEvery;

30     }

    public void run() {
        System.out.printf("=== MIXED LOAD TEST: duration=%d ms
            ===%n", durationMs);
        Random rnd = new Random();
        ConcurrentMap<String,String> reference = new
35         ConcurrentHashMap<>();

        long end = System.currentTimeMillis() + durationMs;
        long ops = 0;

        while (System.currentTimeMillis() < end) {
            String key = "key-" + rnd.nextInt(10_000);

            // PUT
            String value = UUID.randomUUID().toString();
45         long t0 = System.nanoTime();
            storage.put(key, value);
            totalPutNs.addAndGet(System.nanoTime() - t0);
            puts.incrementAndGet();
            reference.put(key, value);

```

```

50      // GET
      t0 = System.nanoTime();
      String got = storage.get(key);
      totalGetNs.addAndGet(System.nanoTime() - t0);
55      gets.incrementAndGet();

      if (++ops % checkEvery == 0) {
          String exp = reference.get(key);
          String act = storage.get(key);
60          if (exp != null && !Objects.equals(exp, act))
              {
                  System.err.printf("Mismatch key=%s
                                     expected=%s actual=%s%n", key, exp, act
                                     );
                  mismatches.incrementAndGet();
              }
      }
65  }

      // Итоги
      double avgPutMs = totalPutNs.get()/1_000_000.0/puts.
        get();
      double avgGetMs = totalGetNs.get()/1_000_000.0/gets.
        get();
70

      System.out.println("=== MIXED TEST COMPLETED ===");
      System.out.printf("Duration: %d ms%n", durationMs);
      System.out.printf("Operations: %d (puts=%d, gets=%d)%n
                        ",
                        puts.get()+gets.get(), puts.get(), gets.get())
                        ;
75      System.out.printf("Avg put latency = %.6f ms%n",
                        avgPutMs);
      System.out.printf("Avg get latency = %.6f ms%n",
                        avgGetMs);
      System.out.printf("Mismatches: %d%n%n", mismatches.get
                        ());
      }

80  private static void deleteDataDirectory(File dir) {
      if (!dir.exists()) return;
      File[] files = dir.listFiles();

```

```

        if (files != null) {
            for (File file : files) {
                if (file.isDirectory()) {
                    deleteDataDirectory(file);
                } else {
                    file.delete();
                }
            }
        }
        dir.delete();
    }

    public static void main(String[] args) throws
        InterruptedException {
        int checkEvery = 1_000;

        System.out.println("Deleting ./data");
        deleteDataDirectory(new File("./data"));
        Thread.sleep(100);

        int duration = 60_000;
        new MixedLoadTest(new StorageCoreAsync(1L*1024*1024,
            5), duration, checkEvery).run();

        System.out.println("Deleting ./data");
        deleteDataDirectory(new File("./data"));
        Thread.sleep(100);

        duration = 600_000;
        new MixedLoadTest(new StorageCoreAsync(1L*1024*1024,
            5), duration, checkEvery).run();
    }
}

```

Листинг П1.19

PutOnlyLoadTest.java - нагрузочное тестирование с мониторингом занимаемой памяти, только операция put

```

package ru.choomandco.diplsm;

import ru.choomandco.diplsm.storage.core.StorageCoreAsync;
5 import ru.choomandco.diplsm.storage.interfaces.DipLSMStorage;

import java.io.File;

```

```

import java.util.ArrayList;
import java.util.List;
10 import java.util.Random;
import java.util.UUID;
import java.util.concurrent.atomic.AtomicLong;

public class PutOnlyLoadTest {
15     private final DipLSMStorage storage;
    private final long durationMs;
    private final long checkEvery;

    private final AtomicLong puts = new AtomicLong();
20     private final AtomicLong totalPutNs = new AtomicLong();

    public PutOnlyLoadTest(DipLSMStorage storage, long
        durationMs, long checkEvery) {
        this.storage = storage;
        this.durationMs = durationMs;
25     this.checkEvery = checkEvery;
    }

    public void run() {
        List<Long> memMetrx = new ArrayList<>();
30     System.out.printf("=== PUT-ONLY TEST: duration=%d ms,
        sample every %d ops ===%n",
            durationMs, checkEvery);

        Random rnd = new Random();
        long end = System.currentTimeMillis() + durationMs;
35     long ops = 0;

        System.out.printf("Memory before run: %d MB%n",
            usedMemoryMB());

        while (System.currentTimeMillis() < end) {
40     String key = "key-" + rnd.nextInt(10_000);
        String value = UUID.randomUUID().toString();

        long t0 = System.nanoTime();
        storage.put(key, value);
45     totalPutNs.addAndGet(System.nanoTime() - t0);
        puts.incrementAndGet();
    }
}

```

```

        if (++ops % checkEvery == 0) {
            //long l = usedMemoryMB();
            //System.out.println(l);
            memMetrx.add(usedMemoryMB());
        }
    }

    double avgPutMs = totalPutNs.get()/1_000_000.0/puts.
        get();

    System.out.println("=== PUT-ONLY TEST COMPLETED ===");
    System.out.printf("Duration: %d ms%n", durationMs);
    System.out.printf("Total puts: %d, avg latency = %.6f
        ms%n",
        puts.get(), avgPutMs);
    System.out.printf("Memory after run: %d MB%n%n",
        usedMemoryMB());
    System.out.println("Mem metrics:");
    for (Long l : memMetrx) {
        System.out.println(l);
    }
}

private static void deleteDataDirectory(File dir) {
    if (!dir.exists()) return;
    File[] files = dir.listFiles();
    if (files != null) {
        for (File file : files) {
            if (file.isDirectory()) {
                deleteDataDirectory(file);
            } else {
                file.delete();
            }
        }
    }
    dir.delete();
}

private long usedMemoryMB() {
    Runtime rt = Runtime.getRuntime();
    return (rt.totalMemory() - rt.freeMemory()) / 1024 /
        1024;
}

```

```

public static void main(String[] args) throws
    InterruptedException {
    long checkEvery = 500_000L;

    System.out.println("Deleting ./data");
    deleteDataDirectory(new File("./data"));
    Thread.sleep(100);

    System.gc();
    int duration = 60_000;
    new PutOnlyLoadTest(new StorageCoreAsync(6L*1024*1024,
        3), duration, checkEvery).run();

    System.gc();
    System.out.println("Deleting ./data");
    deleteDataDirectory(new File("./data"));
    Thread.sleep(100);

    duration = 600_000;
    new PutOnlyLoadTest(new StorageCoreAsync(6L*1024*1024,
        3), duration, checkEvery).run();
    }
}

```

Листинг П1.20

docker-compose.yml - конфигурация среды докера

```

services:
  lsm-store:
    build: .
    container_name: lsm-embedded
    deploy:
      resources:
        limits:
          memory: 512M
          cpus: "0.50"

    mem_limit: 512M
    cpus: 0.5
    entrypoint: >
    sh -c '
      if [ "$MODE" = "MIXED" ]; then

```

```

        java $JAVA_OPTS -cp lsm-storage.jar ru.choomandco.
            diplsm.MixedLoadTest $DURATION_MS $CHECK_EVERY;
    else
        java $JAVA_OPTS -cp lsm-storage.jar ru.choomandco.
            diplsm.PutOnlyLoadTest $DURATION_MS $CHECK_EVERY;
    fi
,

tty: true

```

Листинг П1.21

Dockerfile - конфигурация контейнера с тестируемым хранилищем

```

FROM maven:3.5.2-jdk-8 AS builder
WORKDIR /build

5 COPY pom.xml .
  RUN mvn dependency:go-offline -B

  COPY src ./src
  RUN mvn package -DskipTests -B
10 FROM openjdk:8-jre-slim

  ENV JAVA_OPTS="-Xmx512m"
  WORKDIR /app
15 COPY --from=builder /build/target/DipLSM-1.0-SNAPSHOT.jar ./
    lsm-storage.jar
  ENTRYPOINT ["sh","-c","java $JAVA_OPTS -jar lsm-storage.jar ru
    .choomandco.diplsm.MixedLoadTest 60000 1000"]

```

